FORMULAS

This is a compendium of some computations used in the code, presented in (hopefully) a human-readable form. Unprefixed paths are repository paths; **B** is the BMS installation, **I** is `B/Data/Add-OnIsrael`. Units: lb, lbf, ft^2 unless stated; arguments of trigonometric functions are converted to radians. Examples are rounded for reading.

Contents

1. Moon phase and rise/set 1
1.1. Moon phase 1
1.2. Moonrise and moonset 1
2. Takeoff fuel, gross weight and displayed DRAG 3
2.1. Takeoff fuel 3
2.2. Takeoff gross weight 4
2.3. Loadout drag factor 5
3. Atmosphere, aerodynamic drag and fuel flow 6
3.1. Atmosphere 6
3.2. Aircraft table interpolation 8
3.3. Aerodynamic drag and fuel flow 9
4. Liftoff speed (experimental) 11
5. Minimal fuel on board 13
6. Navigation, weather and terrain 16
6.1. DTC offset geometry 16
6.2. Magnetic headings 17
6.3. Departure weather 19
6.4. Terrain and AGL-to-MSL conversion 20
7. Pop-up attack calculator 22
7.1. Bomb range 22
7.2. Stick, tracking and climb profile 23
7.3. Recovery height and clearance 25
7.4. Map points, turn and navigation offsets 26
References 29

1. Moon phase and rise/set

1.1. Moon phase.

Value. Moon phase displayed in the Weather table, selected from the cycle percentage p for the campaign date.

Formula.

$$p = \operatorname{trunc}(100d/29.530588853) \bmod 100.$$

Variables.

- p: integer cycle percentage.
- $d = (\text{date} - \text{2000-01-06}).\text{days}$: whole days since 6 January 2000, where date is the campaign calendar date.

Date: 12 October 2026.

Data.

- date is read from the year/month/day entries in the first four little-endian unsigned 32-bit integers of `<save>.twx`; the first integer is ignored. d is computed from that date and the fixed epoch 6 January 2000 stored as `date0` in `lib/moon.py`.
- p is computed from d using the fixed 29.530588853-day period in `lib/moon.py:moonphase_crude`. Time of day is ignored; p measures cycle progress, not illuminated fraction.
- The displayed phase is selected from p using the thresholds in `lib/moon.py:moonphase_crude`: new 0–4 or 96–99; waxing crescent 5–20; first quarter 21–30; waxing gibbous 31–45; full 46–55; waning gibbous 56–70; third quarter 71–80; waning crescent 81–95.

Example. `I/Campaign/weight_tests.twx`: 2026-10-10, $d = 9774$, $p = 97 \Rightarrow$ displayed phase `New Moon`.

Code. `lib/moon.py`, `moonphase_crude`:

```
113  delta_days = (date - date0).days
114  phase = int(delta_days*100/29.530588853) % 100
```

1.2. **Moonrise and moonset.**

Values. Moonrise and moonset times for the theater centre on the campaign date, displayed in the Weather table as `HH:MMz` (UTC).

Formulas. The lunar-position series follows Meeus [1, Chapter 45], as referenced in `lib/moon.py:133`. Rise/set uses Julian date JD, lunar longitude/latitude (λ, β), obliquity ϵ and Greenwich sidereal angle θ_G. The formula is

$$\lambda = L' + 10^{-6}\left[\sum_{j=1}^{n} E^{|m_j|} a_j \sin(d_j D + m_j M + u_j M' + f_j F) + A_\lambda\right],$$

with the analogous sum for β (without L').

Then

$$\alpha = \operatorname{atan2}(\sin\lambda\cos\epsilon - \tan\beta\sin\epsilon, \cos\lambda), \qquad \delta = \arcsin(\sin\beta\cos\epsilon + \cos\beta\sin\epsilon\sin\lambda),$$

$$h = \arcsin[\sin\phi\sin\delta + \cos\phi\cos\delta\cos(\theta_G + \ell_E - \alpha)], \qquad t_* = t_i + 120\,\frac{|h_i|}{|h_i| + |h_{i+1}|} \quad \text{seconds.}$$

Variables.

- JD: Julian date in days; $T = (JD - 2451545)/36525$ is Julian centuries since J2000.
- λ, β: computed lunar ecliptic longitude and latitude, respectively.
- L', D, F: Moon's mean longitude, mean elongation and argument of latitude, respectively; computed from T using Meeus's fixed polynomial coefficients in `moon_long_lat`.
- M, M': Sun's and Moon's mean anomalies, respectively; also computed from T using Meeus's fixed polynomial coefficients.
- $E = 1 - 0.002516T - 0.0000074T^2$: time-dependent eccentricity correction factor; the numerical coefficients are from Meeus.
- n, j: retained term count and summation index ($j = 1, \ldots, n$); $n = 5$ is the app's accuracy setting.
- a_j: fixed longitude-series amplitude from Meeus, in millionths of a degree, stored in `SIGMA_LR_TERMS`. The latitude series uses its own fixed amplitudes in `SIGMA_B_TERMS`.
- d_j, m_j, u_j, f_j: fixed integer multipliers from Meeus for D, M, M', F, respectively, stored in each row of the corresponding series table.
- A_λ: time-dependent additive longitude correction, computed as `Sl_additive`; its fixed amplitudes 3958, 1962 and 318 are from Meeus.
- ϵ, θ_G: Earth's obliquity and Greenwich sidereal angle, respectively; computed for the sample time from T and JD.
- α, δ: lunar right ascension and declination, respectively.
- ϕ, ℓ_E: observer latitude (north positive) and longitude (east positive), respectively; taken from the BMS theater centre.
- h: geometric lunar altitude.
- i, t_i: index and time (seconds) of the sample starting a two-minute interval containing a horizon crossing: $h_i \le 0 < h_{i+1}$ for moonrise, or $h_i \ge 0 > h_{i+1}$ for moonset.
- t_*: interpolated crossing time.

Data.

- JD, T are computed from the campaign date read from `<save>.twx` and the two-minute sample times generated in `lib/moon.py:moon_rise_set_times`; `date_to_JD` and `date_to_T` in that file perform the conversions.
- ϕ, ℓ_E are read from `Center Latitude` and `Center Longitude` in the active terrain's `NewTerrain/Theater.txt` by `lib/theater_paths.py`. Missing coordinates use the 50 N, 50 W fallback in `lib/brief_render.py`.
- $L', D, F, M, M', E, A_\lambda$ are computed from T and the resulting angles using Meeus's fixed coefficients, stored in `lib/moon.py:moon_long_lat`.
- a_j, d_j, m_j, u_j, f_j are taken from Meeus's longitude-series table, stored as `SIGMA_LR_TERMS` in `lib/moon.py`. The β series uses its own rows from `SIGMA_B_TERMS` in the same file.
- $n = 5$ is the app's `accuracy` argument set in `lib/brief_render.py`; j enumerates the first n table rows.
- ϵ, θ_G are computed from T and JD by `eps` and `sidereal_time_Greenwich` in `lib/moon.py`.
- $\lambda, \beta, \alpha, \delta, h$ are computed from these time, table and observer inputs by `moon_long_lat`, `moon_asc_decl` and `moon_angle_above_horizon` in `lib/moon.py`. h is geometric altitude, with no refraction or parallax correction.
- i, t_i, t_* are obtained from the sampled altitudes and times by `lib/moon.py:moon_rise_set_times`: indexes i in t_i is determined by the sign changes of h; linear interpolation gives the rise/set time t_*.

Example. `I/Campaign/weight_tests.twx`: 2026-10-10; Israel `Theater.txt`: 32 N, 35 E. Rise 03:16:47.9, set 14:53:46.4 UTC (displayed `Moonrise:  03:16z` / `Moonset:  14:53z`).

Code. The excerpts follow $(\lambda, \beta) \to (\alpha, \delta) \to h$ and then find the horizon crossings. In the final function, `tt` holds two-minute UTC sample datetimes; `hh[i]` is the lunar altitude h_i in degrees at `tt[i]`. The lists `rise_ends` and `set_ends` hold indices of crossing intervals; `rise` and `set` hold the interpolated UTC datetimes. The loop variable `n` is a time-sample counter, separate from the periodic-term count n above. The caller in `lib/brief_render.py:395` supplies `accuracy=5` and west-positive longitude $-\ell_E$.

`lib/moon.py`, `moon_long_lat`: compute and return (λ, β).

```
166  l = [(E**(abs(c[1])))*c[4]*sin((c[0]*D + c[1]*M + c[2]*Mp + c[3]*F)) for c in SIGMA_LR_TERMS[0:accuracy]]
167  b = [(E**(abs(c[1])))*c[4]*sin((c[0]*D + c[1]*M + c[2]*Mp + c[3]*F)) for c in SIGMA_B_TERMS[0:accuracy]]
168
169  Sl_additive = (3958*sin(A1)
170                 + 1962*sin(Lp - F)
171                 + 318*sin(A2))
172  Sb_additive = (-2235*sin(Lp)
173                 +382*sin(A3)
174                 +175*sin(A1-F)
175                 +175*sin(A1+F)
176                 +127*sin(Lp - Mp)
177                 -115*sin(Lp+Mp))
178
179  Sl = sum(l) + Sl_additive
180  Sb = sum(b) + Sb_additive
181
182  lambd = (degrees(Lp) + Sl/(10**6)) % 360
183  beta = (Sb/(10**6)) % 360
184
185  return(lambd, beta)
```

`lib/moon.py`, `moon_asc_decl`: convert these coordinates to (α, δ).

```
188  def moon_asc_decl(date, accuracy = 60):
189      (lambd, beta) = [radians(x) for x in moon_long_lat(date, accuracy = accuracy)]
190      T = date_to_T(date)
191      reps = radians(eps(T))
192      alpha = atan2((sin(lambd)*cos(reps)-tan(beta)*sin(reps)),(cos(lambd)))
193      delta = asin(sin((beta))*cos((reps))+cos((beta))*sin((reps))*sin((lambd)))
194      return (degrees(alpha), degrees(delta))
```

`lib/moon.py`, `moon_angle_above_horizon`: use (α, δ) and the observer coordinates to obtain h.

```
197  def moon_angle_above_horizon(date, latitude, longitude, accuracy = 60):
198      rlatitude = radians(latitude)
199      (alpha, delta) = moon_asc_decl(date, accuracy = accuracy)
200      rdelta = radians(delta)
201      H = sidereal_time_Greenwich(date) - longitude - alpha
202      rH = radians(H)
203      return degrees(asin(sin(rlatitude)*sin(rdelta) + cos(rlatitude)*cos(rdelta)*cos(rH)))
```

lib/moon.py, moon_rise_set_times: build tt and hh, select sign changes, and interpolate rise and set.

```
206 def moon_rise_set_times(day, latitude, longitude, accuracy = 60):
207     # compute angle every two minutes
208     date = dt.datetime.combine(day, dt.time.min)
209     tt = [date + n*dt.timedelta(minutes = 2) for n in range(24*30)]
210     hh = [moon_angle_above_horizon(t, latitude = latitude, longitude = longitude, accuracy = accuracy) for t in
        tt]
211     rise_ends = [i for i in range(len(tt)-1) if hh[i] <= 0 and hh[i+1] > 0]
212     set_ends = [i for i in range(len(tt)-1) if hh[i] >= 0 and hh[i+1] < 0]
213     rise = [tt[i] + (tt[i+1] - tt[i])*abs(hh[i])/(abs(hh[i]) +  abs(hh[i+1])) for i in rise_ends]
214     set = [tt[i] + (tt[i+1] - tt[i])*abs(hh[i])/(abs(hh[i]) +  abs(hh[i+1])) for i in set_ends]
215     return (rise, set)
```

2. Takeoff fuel, gross weight and displayed DRAG

2.1. **Takeoff fuel.**

Value. Takeoff fuel for the selected pilot's saved aircraft, displayed as T/O Fuel in the Admin section, rounded to whole lb. Unavailable values show a dash.

Formula.

$$F_{TO} = F_{save} + \sum_s n_s C_s + bC_{CFT}.$$

Variables.

- F_{TO}, F_{save}: computed takeoff fuel and the saved member's signed fuel offset, respectively, in lb.
- s, n_s: occupied station index and its saved store count. The zero-weight internal gun/ammunition entry is omitted.
- C_s: fuel capacity per store, in lb; WCD Strength when Flags & 0x02 is nonzero, otherwise zero.
- b: 1 if the aircraft supports CFTs and the saved member has them fitted, otherwise 0.
- C_{CFT}, C_{int}: CFT and internal fuel capacities, in lb.
- $C_{fit} = C_{int} + \sum_s n_s C_s + bC_{CFT}$: fitted capacity, in lb; the app requires $0 \le F_{TO} \le C_{fit}$.

Data.

- F_{save} is decoded from the member's fuel_initial_raw entry in the embedded .uni of <save>.cam; the array contains four signed little-endian 32-bit offsets. This offset can be negative and is not a literal internal-fuel quantity.
- s, n_s come from the loadout's weapon_ids and weapon_counts in that .uni file. A single loadout is shared; with one per active member, each uses its own. Other loadout counts are rejected.
- C_s is taken from Strength when Flags & 0x02 is nonzero in the theater's Terrdata/objects/Falcon4_WCD.xml, otherwise it is zero. The saved store ID selects the WCD Num row.
- b is computed from the member's byte flag in loaded_cft_raw in the CAM's embedded .uni, together with aero_new2.has_cft from the aircraft's Sim/Acdata/*.txtpb. The saved flag is used only when the aircraft supports CFTs.
- C_{CFT}, C_{int} are read from auxaero.fuel.cft_fuel and airframe.internal_fuel_lbs, respectively, in the aircraft's Sim/Acdata/*.txtpb.
- F_{TO}, C_{fit} are computed from the preceding saved values and capacities in lib/cam/loadout_data.py: build_aircraft_members, after _loadout_data in that file sums $n_s C_s$.

Example. Falcon2-1, flight 2887/member 0 in I/Campaign/weight_tests.cam, uses I/Sim/Acdata/F-16I-52+IAF.txtpb. The save gives $F_{save} = 5920$, $b = 1$ and one TK300 tank (WCD Num=782, Flags=2, Strength=2040). The aircraft file gives $C_{CFT} = 3060$ and $C_{int} = 5920$:

$$F_{TO} = 5920 + 1(2040) + 1(3060) = 11020 \text{ lb}.$$

This equals C_{fit}; Admin displays 11,020.

Code. Here flight is the parsed CAM flight, station is the current saved hardpoint, and weapon is its WCD row. In _loadout_data, external_fuel starts at zero and accumulates $n_s C_s$ for each retained station. Its returned total is passed to build_aircraft_members. There, sim is the parsed aircraft data, index selects the saved member and loaded_cft is b. The helper _weight checks that its argument is a finite, nonnegative number.

lib/cam/loadout_data.py, _loadout_data:

```
67  external_fuel += station.weapon_count * _weight(weapon.fuel_capacity_lb)
```

lib/cam/loadout_data.py, build_aircraft_members:

```
cft_fuel = _weight(sim.cft_fuel_lb) if loaded_cft else 0
# BMS saves a signed offset, not a literal internal-fuel quantity.
# At empty fuel this offset cancels fitted tank and CFT capacities.
fuel = flight.initial_fuel_lb[index] + _weight(external_fuel) + cft_fuel
capacity = _weight(sim.airframe.internal_fuel_lb) + _weight(external_fuel) + cft_fuel
if not 0 <= fuel <= capacity:
    raise ValueError("Planned fuel is outside fitted capacity")
member["takeoff_fuel_lb"] = fuel
```

2.2. **Takeoff gross weight.**

Value. Takeoff gross weight for the selected pilot's saved aircraft, displayed as `Gross Wt` in Admin, rounded to whole lb. Unavailable values show a dash.

Formula.

$$W_{TO} = W_{empty} + F_{TO} + bW_{CFT} + \sum_s (n_s w_s + w_{rack,s} + w_{pylon,s}).$$

Variables.

- W_{TO}: computed takeoff gross weight, in lb.
- W_{empty}, W_{CFT}: clean-aircraft and dry CFT weights, respectively, in lb.
- F_{TO}, b: takeoff fuel and fitted-CFT flag from section 2.1.
- s, n_s: retained occupied station index and its saved store count.
- $w_s, w_{rack,s}, w_{pylon,s}$: dry weight per store, rack and pylon on station s, respectively, in lb. Hardware counts once per occupied station; an explicitly absent rack or pylon contributes zero.

Data.

- W_{empty}, W_{CFT} are read from `airframe.empty_weight_lbs` and `auxaero.fuel.cft_empty_weight`, respectively, in the aircraft's `Sim/Acdata/*.txtpb`.
- F_{TO}, b, s, n_s reuse the fuel calculation and saved member/loadout data from `<save>.cam` described in section 2.1.
- w_s is read from `Weight` in `Terrdata/objects/Falcon4_WCD.xml`, selecting `Num` by the saved store ID. It excludes fuel contents. Internal stores add weight; the zero-weight gun/ammunition entry at station 0 is already accounted for in clean weight and is skipped.
- $w_{rack,s}, w_{pylon,s}$ are read from WCD `Weight` after resolving hardware. The aircraft `.txtpb`'s `auxaero.hardpoints.hardpoint_grp` selects a group in `Terrdata/objects/BmsRack.dat`; the first compatible rack with sufficient `RackStations` capacity is used. Its `RackCT` and `PylonCT` select hardware by WCD `CtIdx`. Compatibility uses `Falcon4_CT.xml MoverDefinitionData` to link WCD `CtIdx` to `Falcon4_SWD.XML Num/WpnClass`; these XML files share the objects directory. Unresolved hardware leaves gross weight unavailable.
- W_{TO} is computed by summing the preceding fuel, airframe, CFT, store and hardware contributions in `lib/cam/loadout_data.py:build_aircraft_members`; the store/hardware sum is computed by `_loadout_data` in that file.

Example. For the same Falcon2-1 loadout, WCD store rows 158, 256, 782, 179 and 181 give four A120Bs, four CB52Bs, one TK300, one AAQ13 and one AAQ28:

$$4(338) + 4(785) + 392 + 429 + 441 = 5754 \text{ lb}.$$

Selected hardware rows 661, 585, 780, 779 and 622 contribute

$$2(95) + 2(369) + 2(320) + 172 + 42 = 1782 \text{ lb}.$$

With $W_{empty} = 22000$, $W_{CFT} = 1700$, $b = 1$ and $F_{TO} = 11020$:

$$W_{TO} = 22000 + 11020 + 1700 + (5754 + 1782) = 42256 \text{ lb}.$$

Admin displays `42,256`.

Code. In `_loadout_data`, `dry_weight` starts at zero. The resolver supplies the station's rack and pylon; the returned sum is passed to `build_aircraft_members`. There, `clean` is W_{empty}, `cft_dry` is bW_{CFT} and `fuel` is F_{TO} from section 2.1. The remaining names and validation helper are as above.

`lib/cam/loadout_data.py`, `_loadout_data`:

```
74  hardware = resolve_loadout_hardware(
75      flight.support, aircraft.vehicle.number, station.hardpoint_index,
76      station.weapon_id, station.weapon_count,
77  )
78  dry_weight += (station.weapon_count * _weight(weapon.weight_lb)
79                 + _weight(hardware.rack.weight_lb) + _weight(hardware.pylon.weight_lb))
```

`lib/cam/loadout_data.py`, `build_aircraft_members`:

```
164 cft_dry = _weight(sim.fuel_system.get("cft_empty_weight")) if loaded_cft else 0
165 clean = _weight(sim.airframe.empty_weight_lb)
166 if clean == 0:
167     raise ValueError("Missing clean weight")
168 member["gross_weight_lb"] = clean + _weight(dry_weight) + cft_dry + fuel
```

2.3. **Loadout drag factor.**

Value. The selected pilot's saved loadout drag factor, displayed as `DRAG` in Admin as a whole, dimensionless number. Unavailable values show a dash.

Formula.

$$DI = 1 + bDI_{CFT} + \sum_{s \text{ external}} (n_s DI_s + DI_{rack,s} + DI_{pylon,s}).$$

Variables.

- DI: computed displayed drag factor; 1 is the fixed clean-aircraft display baseline used by the app and checked against BMS.
- b, DI_{CFT}: fitted-CFT flag from section 2.1 and the aircraft's CFT drag increment, respectively.
- s, n_s: occupied external station index and its saved store count. A station is external when its `is_external_hardpoint` flag is 1.
- $DI_s, DI_{rack,s}, DI_{pylon,s}$: fixed WCD drag indices per store, rack and pylon, respectively. Hardware counts once per external station.

Data.

- $DI_s, DI_{rack,s}, DI_{pylon,s}$ are read from `Drag` in `Terrdata/objects/Falcon4_WCD.xml`; store IDs and hardware selection use the same files and rules as section 2.2.
- s, n_s come from the saved loadout in `<save>.cam`, restricted to stations whose `auxaero.hardpoints.is_external_hardpoint.values` entry is 1 in the aircraft's `Sim/Acdata/*.txtpb`. Internal stores and their hardware contribute no drag to the sum.
- b is obtained from the saved CFT flag and aircraft support flag as in section 2.1; DI_{CFT} is read from `auxaero.fuel.cft_drag` in the aircraft's `Sim/Acdata/*.txtpb`. This increment is added only when CFTs are fitted.
- The baseline 1 in DI is a fixed display convention in `lib/cam/loadout_data.py:build_aircraft_members`, checked against BMS's clean-loadout display. It is not an aerodynamic drag coefficient or an aircraft-file field.
- DI is computed from the external store/hardware sum, baseline and fitted CFT increment in `lib/cam/loadout_data.py`. Missing drag data leaves DRAG unavailable independently of otherwise known fuel and weight.

Example. All retained Falcon2-1 stations in this save are external. The same store and hardware WCD rows listed in section 2.2 give, respectively,

$$4(4) + 4(22) + 17 + 22 + 22 = 165, \qquad 2(2) + 2(24) + 2(14) + 16 + 3 = 99.$$

The aircraft file gives $DI_{CFT} = 20$, hence

$$DI = 1 + 1(20) + 165 + 99 = 285.$$

Admin displays `285`, matching the BMS loadout-screen reading. Removing CFTs gives 265; a clean loadout without CFTs gives 1.

Code. In `_loadout_data`, `drag_index` starts at zero and accumulates the external store/hardware sum; `external` contains the station flags. The `hardware` objects were resolved in section 2.2. The helper `_drag_index` validates a nonnegative integer. In `build_aircraft_members`, the returned raw sum is augmented with the clean baseline and fitted CFT increment before assignment to `member["drag_index"]`.

`lib/cam/loadout_data.py`, `_loadout_data`:

```
external = aircraft.simulation_data.external_hardpoints
if not 0 <= station.hardpoint_index < len(external):
    raise ValueError("Missing external hardpoint flag")
flag = external[station.hardpoint_index]
if flag not in (0, 1):
    raise ValueError("Invalid external hardpoint flag")
if flag:
    # Raw loadout DI: each store, plus one rack/pylon per station.
    # Internal stores are excluded. The displayed baseline and
    # fitted CFT increment are added per member below.
    drag_index += (station.weapon_count * _drag_index(weapon.drag_index)
                   + _drag_index(hardware.rack.drag_index)
                   + _drag_index(hardware.pylon.drag_index))
```

`lib/cam/loadout_data.py`, `build_aircraft_members`:

```
cft_drag = _drag_index(sim.fuel_system.get("cft_drag")) if loaded_cft else 0
member["drag_index"] = 1 + _drag_index(drag_index) + cft_drag
```

3. Atmosphere, aerodynamic drag and fuel flow

This is a technical section required for further calculations. None of the values are displayed in the briefing directly.

3.1. **Atmosphere.**

Values. Mach number, true airspeed and dynamic pressure at a given MSL altitude and calibrated airspeed. They supply the route-performance model below; the liftoff model uses a fixed sea-level reference.

Formulas. Temperature and pressure follow the first two layers of the *U.S. Standard Atmosphere, 1976* [2, Part 1]. The app uses MSL altitude directly as layer height, without converting to geopotential height. The subsonic airspeed and pressure relations follow [3, eqs. (1), (2), (5)–(7) and (12)]:

$$M = \sqrt{\frac{2}{\kappa-1}\left[\left(1+\frac{q_c}{p}\right)^{(\kappa-1)/\kappa}-1\right]},$$

$$V_{TAS} = M\sqrt{\kappa R T}\,\frac{3600}{1852}, \qquad q = \frac{\kappa}{2}pM^2\,\frac{0.09290304}{4.4482216152605}.$$

Variables.

- M, V_{TAS}, q: computed Mach number (dimensionless), true airspeed in knots and dynamic pressure in lbf/ft^2, respectively.
- h, V_{CAS}: input MSL altitude in feet and calibrated airspeed in knots; $z = 0.3048h$ and $v_c = V_{CAS}(1852/3600)$ are their metre and metre-per-second equivalents.
- $R = 287.05287$ J/(kg K), $g = 9.80665$ m/s^2, $\kappa = 1.4$: fixed dry-air gas constant, gravitational acceleration and specific-heat ratio used by the code.
- $T_0 = 288.15$ K, $p_0 = 101325$ Pa: fixed ISA sea-level temperature and pressure.
- $L_T = 0.0065$ K/m, $z_t = 11000$ m: fixed temperature lapse rate below the tropopause and its height; the layer above is isothermal.
- $T = T_0 - L_T \min(z, z_t)$: computed air temperature in K.
- p: computed static pressure in Pa,

$$p = p_0(T/T_0)^{g/(RL_T)}\exp\left[-\frac{g\max(z-z_t,0)}{RT}\right].$$

- q_c: impact pressure in Pa corresponding to the input CAS,

$$q_c = p_0\left[\left(1+\frac{\kappa-1}{2}\frac{v_c^2}{\kappa R T_0}\right)^{\kappa/(\kappa-1)}-1\right].$$

- 0.3048, 1852, 3600: metres per foot, metres per nautical mile and seconds per hour. The pressure conversion uses 0.09290304 m^2/ft^2 and 4.4482216152605 N/lbf.

Data.

- h, V_{CAS} are obtained from the altitude and CAS columns in `B/User/Briefings/briefing.txt`. In `lib/fuel_brief.py:_altitude`, an altitude suffix `M` means thousands of feet; AGL suffix `A` is rejected. Airport altitude instead comes from the negative of the third coordinate in `[STPT]` entries `target_k` of the loaded DTC `<callsign/save>.ini`. Leg midpoint altitudes and speed selection are described in section 5.
- $h = 0$, $V_{CAS} = 100$ are fixed inputs in `lib/liftoff.py` for the liftoff reference; they are independent of the departure field.
- T, p, q_c, M, V_{TAS}, q are computed from these inputs and the fixed constants above by `lib/route_fuel.py: air_state`. No saved weather is used. The helper requires $-500 \le z \le 20000$ m, positive CAS, and both local Mach and the sea-level Mach corresponding to CAS below 1.

Example. The 2026-10-12 11:57:32 export of `B/User/Briefings/briefing.txt`, Weasel7 STPT 3, gives altitude `5.9M` and CAS `315`. Thus $h = 5900$ ft, $z = 1798.32$ m and $v_c = 162.05$ m/s. The formulas give

$$T = 288.15 - 0.0065(1798.32) = 276.46092 \text{ K}, \qquad p = 81506.13 \text{ Pa}, \quad q_c = 17017.04 \text{ Pa},$$

$$M = 0.527587, \qquad V_{TAS} = 341.836 \text{ kt}, \qquad q = 331.681 \text{ lbf/ft}^2.$$

Code. The arguments `altitude_ft` and `cas_kt` are h, V_{CAS}; after conversion, `altitude` and `cas` are z, v_c. The numeric factors $0.2, 3.5, 5, 2/7, 0.7$ are the corresponding expressions in $\kappa = 1.4$. `_number` validates finite numeric inputs.

`lib/route_fuel.py`, `air_state`:

```
def air_state(altitude_ft: float, cas_kt: float) -> tuple[float, float, float]:
    """Return Mach, TAS in knots and dynamic pressure in lbf/ft² (subsonic)."""
    altitude = _number(altitude_ft, "MSL altitude") * .3048
    cas = _number(cas_kt, "CAS", positive=True) * 1852 / 3600
    if not -500 <= altitude <= 20000:
        raise FuelModelError("MSL altitude range")
    temperature = 288.15 - .0065 * min(altitude, 11000)
    pressure = 101325 * (temperature / 288.15) ** (9.80665 / (287.05287 * .0065))
    if altitude > 11000:
        pressure *= math.exp(-9.80665 * (altitude - 11000) / (287.05287 * temperature))
    sea_mach = cas / math.sqrt(1.4 * 287.05287 * 288.15)
    impact_pressure = 101325 * ((1 + .2 * sea_mach ** 2) ** 3.5 - 1)
    mach = math.sqrt(5 * ((impact_pressure / pressure + 1) ** (2 / 7) - 1))
    if sea_mach >= 1 or mach >= 1:
        raise FuelModelError("Subsonic CAS range")
    tas = mach * math.sqrt(1.4 * 287.05287 * temperature) * 3600 / 1852
    q = .7 * pressure * mach ** 2 * .09290304 / 4.4482216152605
    return mach, tas, q
```

3.2. **Aircraft table interpolation.**

Value. An aircraft-table value at a requested pair of coordinates, after applying the table's multiplier. The same helper supplies lift, drag, correction, thrust and fuel-flow values to the performance models.

Formula.

$$A(x, y) = (1-u)(1-v)A_{00} + u(1-v)A_{10} + (1-u)vA_{01} + uvA_{11}.$$

Variables.

- $A(x, y)$: interpolated value, in the table's units; for base lift, $A = C_L$ is the dimensionless lift coefficient.
- x, y: query coordinates, such as Mach M and angle of attack α in degrees for lift, or altitude in feet and Mach for engine tables.
- x_0, x_1, y_0, y_1: adjacent axis breakpoints enclosing the query; queries outside the available axes are rejected.
- $u = (x - x_0)/(x_1 - x_0)$, $v = (y - y_0)/(y_1 - y_0)$: interpolation fractions within the two intervals.
- $A_{ij} = \mu a_{ij}$: scaled corner values. Here a_{ij} are raw table entries, μ is the multiplier (taken from BMS data; 1 if absent), and $i, j \in \{0, 1\}$ select the lower/upper corner on each axis.

Data.

- a_{ij}, μ are taken from a table's `data.values` rows and `multiplier` in the active aircraft's `Sim/Acdata/*.txtpb`. `lib/cam/opencam/aircraft_data.py:_table` parses them; `lib/fuel.py:_Table.read` applies the multiplier once.
- x_0, x_1, y_0, y_1 are selected from `x_axis.breakpoints`/`y_axis.breakpoints` in that table. If an axis is absent, the caller supplies fallback breakpoints from the same aircraft file: for example, `aero_new2.mach_bp.breakpoints` and `aero_new2.alpha_bp.breakpoints` for base lift/drag. The selections for other tables are listed below and in section 4.
- x, y are query arguments computed from the flight state in `lib/route_fuel.py` or `lib/liftoff.py`. `lib/fuel.py:_bracket` computes the enclosing intervals and u, v; `_Table.at` in that file computes $A(x, y)$.

Example. In `I/Sim/Acdata/F-16C-40IAF.txtpb`, `aero_new2.clift_table` has multiplier 1. Between Mach $0.5, 0.6$ and angles $5°, 6°$, its corner values are

$$(A_{00}, A_{10}, A_{01}, A_{11}) = (0.364, 0.379, 0.441, 0.453).$$

For a chosen query $M = 0.55$, $\alpha = 5.5°$, $u = v = 0.5$:

$$C_L(0.55, 5.5) = \tfrac{1}{4}(0.364 + 0.379 + 0.441 + 0.453) = 0.40925.$$

Code. `_bracket` returns an axis interval index and its interpolation fraction, using the final interval at the upper endpoint. In `read`, `table` holds the parsed raw rows and multiplier; `x,y` are fallback axes. In `at`, they are query coordinates, and `fx,fy` are u, v; `a,b` interpolate along each row. The helpers `_axis` and `_number` validate the inputs.

`lib/fuel.py`, `_bracket`:

```
51  def _bracket(axis: tuple[float, ...], value: float) -> tuple[int, float]:
52      if not axis[0] <= value <= axis[-1]:
53          raise FuelModelError("Aircraft table range")
54      index = min(max(bisect_right(axis, value) - 1, 0), len(axis) - 2)
55      return index, (value - axis[index]) / (axis[index + 1] - axis[index])
```

`lib/fuel.py`, `_Table.read/at`:

```
65  def read(cls, table: AircraftTable | None, x=None, y=None) -> "_Table":
66      if table is None:
67          raise FuelModelError("Required aircraft performance table is missing.")
68      xs = _axis(table.x_breakpoints if table.x_breakpoints is not None else x, "x")
69      ys = _axis(table.y_breakpoints if table.y_breakpoints is not None else y, "y")
70      if len(table.rows) != len(xs) or any(len(row) != len(ys) for row in table.rows):
71          raise FuelModelError("Aircraft table dimensions do not match its breakpoints.")

73      scale = 1.0 if table.multiplier is None else _number(table.multiplier, "table multiplier", positive=True)
74      rows = tuple(tuple(_number(v, "table value") * scale for v in row) for row in table.rows)
75      return cls(xs, ys, rows)
76
77  def at(self, x: float, y: float) -> float:
78      i, fx = _bracket(self.x, x)
79      j, fy = _bracket(self.y, y)
80      a = self.rows[i][j] * (1 - fy) + self.rows[i][j + 1] * fy
81      b = self.rows[i + 1][j] * (1 - fy) + self.rows[i + 1][j + 1] * fy
82      return a * (1 - fx) + b * fx
```

3.3. **Aerodynamic drag and fuel flow.**

Values. Angle of attack, aerodynamic drag and required thrust at the current weight and planned flight state. The model returns whole-aircraft fuel flow and horizontal airspeed for the MFOB integration in section 5.

Formulas. Lift and drag use the coefficient definitions and steady straight-flight force balance in [4, pp. 1–1, 1–4 and 1–5]. Thrust is treated as parallel to the flight path. The engine-table scaling and fuel-flow interpolation are app approximations.

$$\alpha = \alpha_i + \frac{C_L^* - C_{L,i}}{C_{L,i+1} - C_{L,i}}(\alpha_{i+1} - \alpha_i),$$

$$D = qS\,[C_D(M, \alpha)P_D(0, \alpha) + \Delta C_D], \qquad T_{req} = D + W \sin\gamma,$$

$$Q = Q_a + \max\left(0, \frac{T_{req} - T_a}{T_b - T_a}\right)(Q_b - Q_a), \qquad V_h = V_{TAS}\cos\gamma.$$

Variables.

- h, M, q, V_{TAS}: MSL altitude in feet, Mach number, dynamic pressure in lbf/ft^2 and TAS in knots from section 3.1. Engine tables use h, not its metre equivalent z.
- W, S: current gross weight treated as weight in lbf, and the aircraft's wing area in ft^2.
- $s = \Delta h/\Delta d$, $\gamma = \arctan s$: leg gradient and flight-path angle in radians; $\Delta h, \Delta d$ are the altitude change and horizontal distance, both in feet.
- $C_L^* = W\cos\gamma/(qS)$: lift coefficient required for force balance normal to the flight path.
- C_L, C_D, P_L, P_D: dimensionless base lift/drag coefficients and their sideslip correction multipliers. Corrections use zero sideslip and default to 1 when their table is absent.
- α, α_i: computed angle of attack and sampled angles, in degrees. Samples include the base lift-table breakpoints and endpoints of its common range with the drag table, restricted to $[-5°, 15°]$.
- $C_{L,i} = C_L(M, \alpha_i)P_L(0, \alpha_i)$: corrected sampled lift. The samples must increase strictly; i selects the interval enclosing C_L^*, with $C_{L,i} \le C_L^* \le C_{L,i+1}$.
- $DI, \Delta C_D$: takeoff displayed drag factor from section 2.3 and the added stores drag coefficient, $\Delta C_D = (DI - 1)k(M)$. The subtraction removes the clean-display baseline.
- $k(M)$: coefficient increment per drag-index unit,

$$k(M) = k_0 + \operatorname{clip}\left(\frac{M - M_0}{M_1 - M_0}, 0, 1\right)(k_1 - k_0).$$

  M_0, M_1, k_0, k_1 are the Mach endpoints and their coefficients (taken from BMS data); clip clamps the fraction to $[0, 1]$. A constant limiter has $k_0 = k_1$.
- N_e, f_T, f_Q: engine count and dimensionless thrust and fuel-flow scale factors, fixed for the aircraft.
- $A_T(h, \alpha)$: advanced thrust correction multiplier, used when enabled; otherwise 1.
- $r \in \{idle, MIL, AB\}$: engine regime. $T_{table,r}(h, M), Q_{table,r}(h, M)$ are per-engine table values in lbf and lb/h. Whole-aircraft regime values are

$$T_r = N_e f_T A_T(h, \alpha) T_{table,r}(h, M), \qquad Q_r = N_e f_Q Q_{table,r}(h, M).$$

- a, b: lower and upper interpolation regimes: idle/MIL when $T_{req} \le T_{MIL}$, otherwise MIL/AB. Below idle the model uses idle flow; missing required AB data or thrust above AB makes it unavailable.
- D, T_{req}: computed drag and required combined thrust, in lbf. Q, V_h are the returned combined fuel flow in lb/h and horizontal airspeed in knots.

Data.

- h, M, q, V_{TAS} come from section 3.1. s is computed from planned altitude changes and horizontal distances in `lib/route_fuel.py:remaining_route_fuel`; the distances use the first two coordinates of DTC `[STPT] target_k` entries in `<callsign/save>.ini`.
- W is computed from the selected CAM member's takeoff gross weight and fuel, as described in section 2.2, then adjusted for the current required fuel during the backward integration in section 5. DI stays at the saved takeoff value from section 2.3.
- S is read from `airframe.area_sqft` in the aircraft's `Sim/Acdata/*.txtpb`. C_L, C_D are interpolated from `aero_new2.clift_table` and `aero_new2.cdrag_table`, using `mach_bp`/`alpha_bp` as fallback axes. P_L, P_D use `aero_new2.perc_lift_table` and `aero_new2.perc_drag_table`, with `beta_perc_bp`/`alpha_bp` fallback axes. All tables follow section 3.2.
- M_0, k_0, M_1, k_1 are taken, in that order, from `limits.limit.data.values` where `key=LIM_STORES_DRAG` and `type=LIMITER_LINE` in the same aircraft `.txtpb`. For `LIMITER_VALUE`, its single coefficient is used at both endpoints. `lib/route_fuel.py:RoutePerformance.stores_cd` computes ΔC_D.
- N_e, f_T, f_Q are read from `auxaero.engine.n_engines`, `engine.thrust_factor` and `engine.fuel_flow_factor` in that file. $T_{table,r}, Q_{table,r}$ use its `engine.alt_mach_thrust_*_table` and `engine.alt_mach_fuel_flow_*_table`, with `*` replaced by `idle`, `mil` or `ab`. Missing axes inherit the altitude/Mach axes from `engine.alt_mach_thrust_idle_table`.
- A_T is interpolated from `engine.alt_alpha_thrust_factor_table` only when `engine.has_advanced_thrust` is true in that `.txtpb`; its altitude axis falls back to the idle table's, and its angle axis must be supplied by the table.
- $\alpha, D, T_{req}, Q, V_h$ are computed from these inputs in `lib/route_fuel.py:RoutePerformance.flow_from_simulation` in that class requires `fmtype=FM_AFM` and `engine.has_fuel_flow` true in the aircraft file. Out-of-range queries or invalid lift/engine ranges make the route calculation unavailable.

Example. Weasel7, flight 4/member 0 in `I/Campaign/weight_tests.cam`, has takeoff weight 42689 lb and $DI = 200$. Use that weight for this illustrative level-flight sample ($s = \gamma = 0$), with the STPT 3 atmosphere from section 3.1. `I/Sim/Acdata/F-16C-40IAF.txtpb` gives $S = 300$ ft^2 and the limiter values $(0.9, 0.0001, 1.0, 0.000283)$. Since $M = 0.527587 < 0.9$, $k(M) = 0.0001$ and $\Delta C_D = (200 - 1)(0.0001) = 0.0199$. The required lift and enclosing

corrected table samples give

$$C_L^* = \frac{42689}{331.68072(300)} \simeq 0.429017, \qquad \alpha = 5 + \frac{0.429017 - 0.3681381}{0.4443105 - 0.3681381}(6-5) \simeq 5.7992^\circ.$$

Here $P_L = P_D = A_T = 1$; the drag table gives $C_D = 0.04108642$, hence

$$D = T_{req} = 331.68072(300)(0.04108642 + 0.0199) \simeq 6068.406 \text{ lbf}.$$

The aircraft has $N_e = f_T = f_Q = 1$. Its scaled idle/MIL table values are $(T_{idle}, Q_{idle}) = (-443.464, 713.09)$ and $(T_{MIL}, Q_{MIL}) = (13649.320, 11741.231)$:

$$Q = 713.09 + \frac{6068.406 + 443.464}{13649.320 + 443.464}(11741.231 - 713.09) \simeq 5808.88 \text{ lb/h}, \qquad V_h = 341.836 \text{ kt}.$$

The actual MFOB integration reevaluates this model as required fuel and therefore W change.

Code. In `from_simulation`, `sim` is the parsed aircraft data and `engines` is N_e. The returned object stores the wing area, $N_e f_T$, $N_e f_Q$, interpolation tables and stores limiter.

`lib/route_fuel.py`, `RoutePerformance.from_simulation`:

```
return cls(
    _number(sim.airframe.wing_area_sqft, "wing area", positive=True),
    engines * _number(sim.engine.parameters.get("thrust_factor"), "thrust factor", positive=True),
    engines * _number(sim.engine.parameters.get("fuel_flow_factor"), "fuel-flow factor", positive=True),
    tables, values,
)
```

In `stores_cd`, `index` is $DI - 1$ and the limiter tuple is (M_0, k_0, M_1, k_1). In `flow`, the inputs are altitude in feet, CAS in knots, current weight, takeoff DI and leg gradient. `q_area` is qS; `alphas,lifts` hold $\alpha_i, C_{L,i}$. The `_bracket` call in `flow` locates the required lift between samples. The local `engine` helper evaluates and scales a regime table; `t0,t1,f0,f1` are T_a, T_b, Q_a, Q_b. The returned pair is (Q, V_h).

`lib/route_fuel.py`, `RoutePerformance.stores_cd`:

```
def stores_cd(self, mach: float, drag_index: float) -> float:
    index = _number(drag_index, "T/O drag factor") - 1
    if index < 0:
        raise FuelModelError("T/O drag factor")
    x0, y0, x1, y1 = self.stores_drag
    fraction = max(0., min(1., (mach - x0) / (x1 - x0)))
    return index * (y0 + fraction * (y1 - y0))
```

`lib/route_fuel.py`, `RoutePerformance.flow`:

```
mach, tas, q = air_state(altitude_ft, cas_kt)
weight = _number(weight_lb, "gross weight", positive=True)
gamma = math.atan(gradient)
q_area = q * self.wing_area
cl, cd = self.tables["cl"], self.tables["cd"]
low, high = max(-5., cl.y[0], cd.y[0]), min(15., cl.y[-1], cd.y[-1])
alphas = sorted({low, high, *(a for a in cl.y if low < a < high)})

def correction(name, alpha):
    table = self.tables.get(name)
    return table.at(0, alpha) if table else 1.

lifts = tuple(cl.at(mach, a) * correction("perc_lift_table", a) for a in alphas)
if any(a >= b for a, b in zip(lifts, lifts[1:])):
    raise FuelModelError("Cruise lift range")
i, f = _bracket(lifts, weight * math.cos(gamma) / q_area)
alpha = alphas[i] + f * (alphas[i + 1] - alphas[i])
drag = (cd.at(mach, alpha) * correction("perc_drag_table", alpha)
        + self.stores_cd(mach, drag_index)) * q_area
required_thrust = drag + weight * math.sin(gamma)
factor = self.tables.get("alpha_thrust")
thrust_scale = self.thrust_scale * (factor.at(altitude_ft, alpha) if factor else 1.)

def engine(quantity, regime, scale):
    return self.tables[f"{quantity}_{regime}"].at(altitude_ft, mach) * scale

t0, t1 = (engine("thrust", r, thrust_scale) for r in ("idle", "mil"))
f0, f1 = (engine("fuel_flow", r, self.flow_scale) for r in ("idle", "mil"))
if t1 <= t0 or f0 <= 0 or f1 < f0:
```

```
150     raise FuelModelError("Idle/MIL performance")
151 if required_thrust > t1:
152     if not {"thrust_ab", "fuel_flow_ab"} <= self.tables.keys():
153         raise FuelModelError("Afterburner performance")
154     t0, f0 = t1, f1
155     t1, f1 = engine("thrust", "ab", thrust_scale), engine("fuel_flow", "ab", self.flow_scale)
156     if t1 <= t0 or f1 < f0 or required_thrust > t1:
157         raise FuelModelError("Required thrust exceeds full AB")
158 fraction = max(0., (required_thrust - t0) / (t1 - t0))
159 return f0 + fraction * (f1 - f0), tas * math.cos(gamma)
```

4. Liftoff speed (experimental)

Value. The selected pilot's estimated liftoff speed, displayed as `L/O SPD` in Admin, rounded to a whole knot. The tooltip specifies KCAS. Unavailable values show a dash.

Formula. The model assumes sea-level ISA, full afterburner, takeoff flaps and horizontal airflow. The aircraft's takeoff pitch is treated as angle of attack, with thrust along the body axis. Liftoff occurs when lift plus vertical thrust supports takeoff gross weight:

$$C_{L,TO}(M,\alpha) = C_L(M,\alpha)P_L(0,\alpha) + C_{L,TEF}(M,\alpha)\frac{\delta_{TO}}{\delta_{max}},$$

$$q(M_{LO})SC_{L,TO}(M_{LO},\alpha) + T(M_{LO})\sin\alpha = W_{TO},$$

$$V_{LO} = \frac{V_{ref}}{M_{ref}}M_{LO}.$$

The app finds the root by 60 bisection iterations over the Mach range corresponding to 70–330 KCAS, after checking that support increases strictly throughout that range.

Variables.

- V_{LO}, M_{LO}: computed liftoff speed in KCAS and its Mach number; M is a candidate Mach number.
- W_{TO}, S: takeoff gross weight from section 2.2, treated as weight in lbf, and the aircraft's wing area in ft^2.
- α: takeoff pitch in degrees (taken from BMS data), used as a fixed angle of attack; the app requires $0 < \alpha < 30°$.
- $C_{L,TO}, C_L, C_{L,TEF}, P_L$: computed takeoff lift coefficient, base lift coefficient, full trailing-edge-flap lift increment and sideslip lift multiplier, respectively. All are dimensionless; $P_L(0,\alpha)$ uses zero sideslip.
- $\delta_{TO}, \delta_{max}$: takeoff and maximum trailing-edge-flap deflections in degrees. Their ratio is the applied flap fraction, required to lie in $[0,1]$, with $\delta_{max} > 0$.
- $T(M) = N_e f_T T_{AB}(0,M)$: combined engine thrust in lbf. N_e, f_T are the engine count and thrust scale factor; T_{AB} is the per-engine full-afterburner thrust table, evaluated at zero altitude in feet.
- $V_{ref} = 100$ KCAS: fixed reference speed chosen in the code; M_{ref}, q_{ref} are its sea-level ISA Mach number and dynamic pressure, computed using the atmosphere formulas in section 3: $M_{ref} \simeq 0.15117647$, $q_{ref} \simeq 33.85529$ lbf/ft^2.
- $q(M) = q_{ref}(M/M_{ref})^2$: dynamic pressure in lbf/ft^2. The reference scaling applies at the fixed sea-level ISA conditions.

Data.

- W_{TO} is computed from the saved member's fuel and loadout in `<save>.cam` and the aircraft/store weights, using the files and fields listed in section 2.2. `lib/cam/loadout_data.py` passes `gross_weight_lb` to the liftoff model.
- S, α are read from `airframe.area_sqft` and `auxaero.ai_behavior.opt_takeoff_pitch`, respectively, in the aircraft's `Sim/Acdata/*.txtpb`. The parser `lib/cam/opencam/aircraft_data.py` exposes them as `wing_area_sqft` and `takeoff_pitch_deg`.
- $\delta_{TO}, \delta_{max}$ are read from `auxaero.tef.tef_take_off` and `auxaero.tef.tef_max_angle`, respectively, in the same aircraft `.txtpb`.
- $C_L, C_{L,TEF}, P_L$ are interpolated from `aero_new2.clift_table`, `aero_new2.c_lift_tef_table` and `aero_new2.perc_lift_table`, respectively, in that `.txtpb`. Their fallback axes within `aero_new2` are `mach_bp/alpha_bp`, `mach_tef_bp/alpha_tef_bp`, and `beta_perc_bp/alpha_bp`. `lib/fuel.py:_Table` reads `data.values`, applies each table's `multiplier` (1 if absent), and interpolates bilinearly; explicit `x_axis.breakpoints/y_axis.breakpoints` take precedence.
- N_e, f_T are read from `auxaero.engine.n_engines` and `engine.thrust_factor` in the aircraft `.txtpb`. T_{AB} is interpolated from `engine.alt_mach_thrust_ab_table`, using the same table rules. Missing altitude/-Mach axes are taken from `engine.alt_mach_thrust_idle_table` in that file.

- M_{ref}, q_{ref} are computed by `lib/route_fuel.py:air_state` at 0 ft and 100 KCAS, using fixed ISA constants rather than saved weather or airfield elevation. `lib/liftoff.py:LiftoffPerformance` computes $C_{L,TO}$, T and V_{LO} from the preceding data. It requires `fmtype=FM_AFM` in the aircraft file, valid table coverage, increasing support and a root within 70–330 KCAS. `lib/admin_brief.py` rounds the result.

Stores and CFTs enter through gross weight; DRAG is not an input. The estimate omits ground effect, trim/CG, rotation dynamics, separate store/CFT lift increments and advanced thrust corrections.

Example. Falcon2-1 in `I/Campaign/weight_tests.cam` has $W_{TO} = 42256$ lb (section 2.2). Its `I/Sim/Acdata/F-16I-52+IAF.txtpb` gives $S = 300$ ft^2, $\alpha = 12°$, $\delta_{TO} = \delta_{max} = 21.5°$, $N_e = 1$ and $f_T = 1$. At the computed root $M_{LO} \simeq 0.28798432$, the tables give

$$C_{L,TO} = 0.8730039(1) + 0.1239225(1) \simeq 0.9969264.$$

With $q \simeq 122.85585$ lbf/ft^2 and $T \simeq 26513.7865$ lbf,

$$122.85585(300)(0.9969264) + 26513.7865 \sin 12° \simeq 42256 \text{ lbf}.$$

Thus $V_{LO} = 190.49546$ KCAS; Admin displays `190`.

For comparison, `B/Data/Sim/Acdata/F-15C.txtpb` gives $S = 608$ ft^2, $\alpha = 10°$, $\delta_{TO} = \delta_{max} = 45°$, $N_e = 2$ and $f_T = 1$. With an illustrative $W_{TO} = 50000$ lb, the model finds $C_{L,TO} = 0.7757$, $T = 40982$ lbf and $V_{LO} = 163.88281$ KCAS, displayed as `164`. These are model estimates, not measured liftoff speeds in BMS.

Code. In `from_simulation`, `sim` holds the parsed aircraft data; `cl`, `flap` and `correction` are the three lift tables, and `thrust` is the AB table. `_Table` implements the interpolation described above; `_number` validates numeric inputs. The `mach` nodes combine the lift, flap and thrust Mach breakpoints with the search limits. At each node, `lift` stores $C_{L,TO}$ and `vertical` stores $T \sin \alpha$. The resulting object stores $q_{ref} S / M_{ref}^2$ as `q_area_per_mach2`.

`lib/liftoff.py`, reference atmosphere:

```
_REFERENCE_MACH, _, _REFERENCE_Q = air_state(0, 100)
_KCAS_PER_MACH = 100 / _REFERENCE_MACH
```

`lib/liftoff.py`, `LiftoffPerformance.from_simulation`:

```
area = _number(sim.airframe.wing_area_sqft, "wing area", positive=True)
alpha = _number(sim.takeoff_pitch_deg, "takeoff pitch", positive=True)
if alpha >= 30:
    raise ValueError("Takeoff pitch range")
tef = sim.flaps.get("tef", {})
flap_fraction = (_number(tef.get("tef_take_off"), "takeoff TEF")
                 / _number(tef.get("tef_max_angle"), "maximum TEF", positive=True))
if not 0 <= flap_fraction <= 1:
    raise ValueError("Takeoff TEF range")
aero = sim.aerodynamics.get("aero_new2")
if aero is None:
    raise ValueError("Liftoff lift tables required")
cl = _Table.read(aero.tables.get("clift_table"),
                 aero.breakpoints.get("mach_bp"), aero.breakpoints.get("alpha_bp"))
flap = _Table.read(aero.tables.get("c_lift_tef_table"),
                   aero.breakpoints.get("mach_tef_bp"), aero.breakpoints.get("alpha_tef_bp"))
correction = _Table.read(aero.tables.get("perc_lift_table"),
                         aero.breakpoints.get("beta_perc_bp"), aero.breakpoints.get("alpha_bp"))
lift_factor = correction.at(0, alpha)
idle = _Table.read(sim.engine.tables.get("alt_mach_thrust_idle_table"))
thrust = _Table.read(sim.engine.tables.get("alt_mach_thrust_ab_table"), idle.x, idle.y)
scale = (_number(sim.engine.number_of_engines, "engine count", positive=True)
         * _number(sim.engine.parameters.get("thrust_factor"), "thrust factor", positive=True)
         * math.sin(math.radians(alpha)))
low, high = (v / _KCAS_PER_MACH for v in (70., 330.))
mach = tuple(sorted({low, high, *(m for m in (*cl.x, *flap.x, *thrust.y) if low < m < high)}))
lift = tuple(_number(cl.at(m, alpha) * lift_factor + flap.at(m, alpha) * flap_fraction,
                     "takeoff lift coefficient", positive=True) for m in mach)
vertical = tuple(_number(thrust.at(0, m) * scale, "vertical AB thrust", positive=True) for m in mach)
result = cls(_REFERENCE_Q / _REFERENCE_MACH ** 2 * area, mach, lift, vertical)
```

The omitted validation checks that the derivative of support is positive on every Mach interval. In `support`, `_bracket` returns the interval index and interpolation fraction; `thrust` there is already the vertical component. The returned force is compared with W_{TO} in `speed_kcas`, which converts the final Mach to KCAS.

`lib/liftoff.py`, `LiftoffPerformance.support/speed_kcas`:

```
def support(self, mach: float) -> float:
    i, fraction = _bracket(self.mach, mach)
    cl = self.lift[i] + fraction * (self.lift[i + 1] - self.lift[i])
    thrust = self.vertical_thrust[i] + fraction * (self.vertical_thrust[i + 1] - self.vertical_thrust[i])
    return self.q_area_per_mach2 * mach ** 2 * cl + thrust

def speed_kcas(self, gross_weight_lb: float) -> float:
    weight = _number(gross_weight_lb, "takeoff gross weight", positive=True)
    low, high = self.mach[0], self.mach[-1]
    if not self.support(low) <= weight <= self.support(high):
        raise ValueError("Liftoff speed outside 70-330 KCAS")
    for _ in range(60):
        middle = (low + high) / 2
        if self.support(middle) < weight:
            low = middle
        else:
            high = middle
    return (low + high) / 2 * _KCAS_PER_MACH
```

5. Minimal fuel on board

Minimal fuel on board (MFOB) is displayed in the flightplan. It is the minimal amount of fuel required to fly the rest of the flight plan from the current steerpoint + 500 lb.

Value. The selected pilot's required fuel on arrival at each steerpoint, displayed as `MFOB` in the flightplan, in lb rounded up to the next 100 lb. It includes any explicitly timed hold at that point. Unavailable values and rows beyond the first planned landing show a dash.

Formulas. Start with $F = F_{land}$ at the first planned landing and work backward. At each steerpoint, add its hold requirement before recording F_i, then integrate the preceding leg. Each step adds fuel using a midpoint estimate of fuel weight:

$$\begin{aligned} Q_1 &= Q(h, V_{CAS}, W_{dry} + F, DI, s), \\ Q_2 &= Q(h, V_{CAS}, W_{dry} + F + Q_1\Delta t/2, DI, s), \\ F_{new} &= F + Q_2\Delta t, \\ \mathrm{MFOB}_i &= 100\,\lceil \mathrm{round}(F_i/100, 9) \rceil\,. \end{aligned}$$

Continue with the unrounded F_{new}. The landing reserve is included once; the upward rounding is applied only to displayed cells.

Variables.

- i, L: route-point index and first landing index, with $i = 1, \ldots, L$. For $i \geq 2$, leg i is flown from $i-1$ to i.
- F_i, MFOB_i: unrounded and displayed required fuel at arrival at point i, in lb, including its hold and the landing reserve.
- F, F_{new}: fuel requirement before and after one backward integration step, in lb. Adding a step moves the estimate earlier along the flight.
- $F_{land} = 500$ lb: fixed landing reserve. The 100 lb display increment and nine decimal places in round are fixed app settings. $\mathrm{round}(x, 9)$ rounds x to nine decimal places before the ceiling $\lceil\cdot\rceil$, suppressing floating-point noise.
- W_{TO}, F_{TO}: the saved member's takeoff gross weight and fuel from sections 2.2 and 2.1. $W_{dry} = W_{TO} - F_{TO}$ is its fixed non-fuel weight, including retained stores, tanks and CFT dry mass. All weights are in lb.
- DI: fixed takeoff drag factor from section 2.3.
- N_i, E_i, Z_i: DTC coordinates in feet; N_i, E_i are north/east, and $-Z_i$ supplies airport elevation. $h_i, V_{CAS,i}$ are the planned MSL altitude in feet and CAS in knots assigned to point i.
- d_i: horizontal leg length in NM, with $c_{ft} = 1852/0.3048$ feet per NM:

$$d_i = \frac{\sqrt{(N_i - N_{i-1})^2 + (E_i - E_{i-1})^2}}{c_{ft}}.$$

- $n_i = \max(1, \lceil d_i/2 \rceil)$: leg step count, giving $\Delta d = d_i/n_i \leq 2$ NM. $j = 0, \ldots, n_i - 1$ counts steps backward from destination i.
- h, V_{CAS}, s: altitude in feet, CAS in knots and forward flight-path gradient used for a leg step:

$$h = h_{i-1} + \left(1 - \frac{j + 1/2}{n_i}\right)(h_i - h_{i-1}), \qquad V_{CAS} = V_{CAS,i}, \qquad s = \frac{h_i - h_{i-1}}{c_{ft} d_i}.$$

Thus both flow evaluations use the same spatial midpoint. The gradient retains the forward climb/descent sign during backward integration.

- $Q(\cdot), Q_1, Q_2$: whole-aircraft fuel-flow model from section 3.3, and its first and midpoint-weight evaluations, in lb/h. V_h is the horizontal airspeed in knots returned with Q_1; a leg step uses $\Delta t = \Delta d / V_h$ hours.
- $t_{arrival,i}, t_{departure,i}$: arrival/departure clock times in seconds after midnight. The hold duration in hours is

$$H_i = \frac{(t_{departure,i} - t_{arrival,i}) \bmod 86400}{3600}.$$

The modulo allows a hold to cross midnight. Points without a timed hold, including takeoff and landing, have $H_i = 0$.

- $n_{H,i} = \max(1, \lceil 120 H_i \rceil)$: step count for a nonzero hold, giving $\Delta t = H_i / n_{H,i} \le 1/120$ h (30 s). Hold steps use $h = h_i$, $V_{CAS} = V_{CAS,i}$ and $s = 0$ in the same fuel-weight update.

Data.

- W_{TO}, F_{TO}, DI come from the matched aircraft member in `<save>.cam`, using the loadout data and files described in section 2. `lib/fuel_brief.py:build_fuel_plan` computes W_{dry} from `gross_weight_lb` minus `takeoff_fuel_lb`. Matching uses the briefing's package and flight/callsign, then the selected pilot's roster position.
- N_i, E_i, Z_i, L come from the loaded DTC `<callsign/save>.ini`, `[STPT]` entries `target_k=N,E,Z,action,...`, where $i = k + 1$. `lib/parsers/parse_callsign_ini.py` parses these entries; `lib/fuel_brief.py:_route_points` stops at the lowest steerpoint number with action 7 or 27. All points from 1 through L are required.
- $h_i, V_{CAS,i}$ come from the altitude and CAS columns of `B/User/Briefings/briefing.txt`, parsed by `lib/parsers/parse_briefing_txt.py`. An altitude suffix `M` means thousands of feet; `A` (AGL) is unsupported. Takeoff and landing use $h_i = -Z_i$ from the DTC instead. A leg uses its destination row's CAS; takeoff borrows STPT 2's CAS, and an absent landing row borrows the preceding row's.
- H_i is computed by `lib/fuel_brief.py:_hold_hours` from the briefing row's arrival time and `Departure:` time in a hold/departure comment. A patrol/attack label alone adds no time; an explicit hold/departure comment with missing or invalid times makes the calculation unavailable. Leg duration instead comes from DTC distance and model airspeed.
- Q, V_h are computed from the flight state, current fuel weight and takeoff DI by `lib/route_fuel.py:RoutePerformance.flow`, using the aircraft's `Sim/Acdata/*.txtpb` fields described in section 3.3.
- F_{land}, the display increment and c_{ft} are `ALLOWANCE_LB`, `ROUNDING_LB` and `FEET_PER_NM` in `lib/fuel.py`. The 2 NM/30 s step limits, nine-place rounding and backward updates of F are implemented in `lib/route_fuel.py:remaining_route_fuel`. A zero-length leg at unchanged altitude is skipped; a zero-length altitude transition is rejected.
- MFOB$_i$ is attached to each briefing row by `lib/fuel_brief.py:build_fuel_plan` and formatted with thousands separators in `templates/flightplan.html`. Invalid route, matching or performance data makes the planned values unavailable.

Store mass and DI stay at takeoff values while fuel weight changes. The estimate includes no wind, acceleration, bank penalty, refuelling credit or store releases; altitude transitions are linear.

Example. Use `I/Campaign/weight_tests.cam`, `I/Campaign/weight_tests.ini` and the 2026-10-12 11:57:32 export of `B/User/Briefings/briefing.txt`: Weasel7, flight 4/member 0, with `I/Sim/Acdata/F-16C-40IAF.txtpb`. The saved member gives

$$W_{dry} = 42689 - 9202 = 33487 \text{ lb}, \qquad DI = 200.$$

The first landing is STPT 10, where $F_{10} = 500$ lb. Working backward through the later legs reaches STPT 3's departure with $F = 6331.6719$ lb. Its arrival/departure times, 07:28:24–07:34:24, give

$$H_3 = \frac{27264 - 26904}{3600} = 0.1 \text{ h}, \qquad n_{H,3} = \lceil 120(0.1) \rceil = 12, \qquad \Delta t = \frac{0.1}{12} = \frac{1}{120} \text{ h}.$$

At 5900 ft / 315 KCAS and $s = 0$, the first backward hold step gives $Q_1 = 5655.4490$ and $Q_2 = 5656.7086$ lb/h:

$$F_{new} = 6331.6719 + \frac{5656.7086}{120} \simeq 6378.8112 \text{ lb}.$$

After all 12 steps, the hold adds 567.0591 lb and $F_3 = 6898.7310$ lb; the displayed MFOB is `6,900`. Continuing backward gives:

STPT	1	2	3	4	5	6	7	8	9	10
MFOB (lb)	7,400	7,000	6,900	6,200	3,600	3,500	3,100	2,700	600	500

STPT 11 is the alternate landing and displays a dash.

Code. In `build_fuel_plan`, `member` is the matched saved aircraft, `model` is its `RoutePerformance` object, and `route` contains the parsed `RoutePoint` records through the first landing. The difference `gross - takeoff_fuel` is W_{dry}.

`lib/fuel_brief.py`, `build_fuel_plan`:

```
    gross = _planned_number(member.get("gross_weight_lb"), "T/O gross weight")
    takeoff_fuel = _planned_number(member.get("takeoff_fuel_lb"), "T/O fuel")
    if takeoff_fuel < 0:
        raise ValueError("T/O fuel")
    route, landing_number = _route_points(rows, dtc)
    values = remaining_route_fuel(model, route, gross - takeoff_fuel, member.get("drag_index"))
```

In `_hold_hours`, `row` is a parsed briefing steerpoint. `arrival[1]` and `departure[1]` are the matched clock strings; the local `seconds` helper converts them to seconds after midnight.

`lib/fuel_brief.py`, `_hold_hours`:

```
def _hold_hours(row: Any) -> float:
    comment = str(_attr(row, "comment", ""))
    # Timed patrol/attack repetition can also carry a Departure annotation.
    # Approximate that dwell at this point's planned speed and altitude.
    if not re.search(r"\b(?:hold|departure)\b", comment, re.IGNORECASE):
        return 0.
    departure = re.search(r"Departure:\s*(\d{2}:\d{2}:\d{2})z?", comment, re.IGNORECASE)
    arrival = re.fullmatch(r"(\d{2}:\d{2}:\d{2})z?", str(_attr(row, "time", "")).strip(), re.IGNORECASE)
    if not departure or not arrival:
        raise ValueError("Hold timing")

    def seconds(text):
        h, m, s = map(int, text.split(":"))
        if h > 23 or m > 59 or s > 59:
            raise ValueError("Hold timing")
        return h * 3600 + m * 60 + s

    return ((seconds(departure[1]) - seconds(arrival[1])) % 86400) / 3600
```

In `remaining_route_fuel`, `required` is F and `dry` is W_{dry}. `point` is destination i and `previous` is $i-1$; Python's `index` is zero-based. `steps`, `step`, `ds` and `dt` correspond to n_i, j, Δd and Δt for a leg; a hold uses $n_{H,i}$ instead. `first,middle` are Q_1, Q_2. `result[point.number]` records the rounded requirement after adding the hold, while `required` retains full precision.

`lib/route_fuel.py`, `remaining_route_fuel`:

```
    dry = _number(dry_weight_lb, "dry weight", positive=True)
    if not points:
        raise FuelModelError("Planned route")
    required = float(ALLOWANCE_LB)
    result = {}

    def rounded():
        return ROUNDING_LB * math.ceil(round(required / ROUNDING_LB, 9))

    for index in range(len(points) - 1, -1, -1):
        point = points[index]
        hold = _number(point.hold_hours, "hold duration")
        if hold < 0 or hold > 24:
            raise FuelModelError("Hold duration")
        if hold:
            steps = max(1, math.ceil(hold * 120))
            dt = hold / steps
            for _ in range(steps):
                first, _ = model.flow(point.altitude_ft, point.cas_kt, dry + required, drag_index)
                middle, _ = model.flow(point.altitude_ft, point.cas_kt,
                                       dry + required + first * dt / 2, drag_index)
                required += middle * dt
        result[point.number] = rounded()
        if index == 0:
            break
        previous = points[index - 1]
        distance = math.hypot(point.north_ft - previous.north_ft,
                              point.east_ft - previous.east_ft) / FEET_PER_NM
        if distance == 0:
```

```
209          if point.altitude_ft != previous.altitude_ft:
210              raise FuelModelError("Zero-length altitude transition")
211          continue
212      gradient = (point.altitude_ft - previous.altitude_ft) / (distance * FEET_PER_NM)
213      steps = max(1, math.ceil(distance / 2))
214      ds = distance / steps
215      for step in range(steps):
216          fraction = 1 - (step + .5) / steps
217          altitude = previous.altitude_ft + fraction * (point.altitude_ft - previous.altitude_ft)
218          first, speed = model.flow(altitude, point.cas_kt, dry + required, drag_index, gradient)
219          dt = ds / speed
220          middle, _ = model.flow(altitude, point.cas_kt, dry + required + first * dt / 2,
221                                 drag_index, gradient)
222          required += middle * dt
223  return result
```

6. Navigation, weather and terrain

6.1. **DTC offset geometry.**

Values. DTC offset-marker positions on the map, and the `BRG`/`RNG` values updated when an offset is dragged: true bearing to 0.1° and horizontal range to the nearest foot.

Formulas.

$$N' = N + r\cos\theta_T, \qquad E' = E + r\sin\theta_T,$$

$$r = \sqrt{\Delta N^2 + \Delta E^2}, \qquad \theta_T = \left(\frac{180^\circ}{\pi}\,\mathrm{atan2}(\Delta E, \Delta N)\right) \bmod 360^\circ.$$

The first pair projects an offset; the second recovers bearing/range from two positions. Geometry uses the theater's north/east plane.

Variables.

- N, E and N', E': origin and offset north/east coordinates, in ft. $\Delta N = N' - N$ and $\Delta E = E' - E$ are their differences.
- r: horizontal range in ft; elevation is excluded.
- θ_T: clockwise bearing from true north, in degrees. $\mathrm{atan2}(\Delta E, \Delta N)$ returns radians; the explicit factor $180^\circ/\pi$ converts to degrees. Projection converts θ_T back to radians before evaluating sine/cosine.

Data.

- N, E are read from the first two fields of `[STPT].target_k=N,E,Z,action,...` in the loaded `<callsign/save>.ini`; the displayed steerpoint number is $k+1$. The parser is `lib/parsers/parse_callsign_ini.py`.
- θ_T, r come from `[NAV OFFSETS]` in the same file, or the edited DTC fields. `VIP`, `VIPPUP`, `VRP` and `VRPPUP` rows store `stpt,brg,rng,elev`; `OA1-stpt`/`OA2-stpt` store `brg,rng,elev`. Here `brg` is true degrees and `rng` is feet.
- N', E' are computed by `projectOffsetFromBmsCoord` or `projectBearingRange` in `templates/dtc.html`. Dragging supplies the new point through `mapLatLngToBmsCoord`; `getDraggedOffsetMetrics` applies the inverse formulas. Pop-up geometry in section 7 uses the same projection and `bearingRangeBetween`.
- The conversion between N, E and web-map latitude/longitude uses the theater's WGS84 Transverse Mercator projection. Its construction follows oakdesign's *Falcon BMS Tacview Converter* [5], as credited in `templates/map.html` (line 646). The DTC reverse conversion is `mapLatLngToBmsCoord` in `templates/dtc.html`.

Example. Using STPT 1 and 2 from `I/Campaign/weight_tests.ini` as the two positions (these are route points, not saved offset rows):

$$(N, E) = (1923708.25, 1733469.375), \qquad (N', E') = (1999147.75, 1687549.75)\ \text{ft}.$$

Thus

$$r = \sqrt{75439.5^2 + (-45919.625)^2} = 88316.08076\ \text{ft}, \qquad \theta_T = 328.67135^\circ.$$

The offset formatter gives `BRG 328.7`, `RNG 88316`. Projecting the unrounded r, θ_T from STPT 1 recovers STPT 2.

Code. `templates/dtc.html`, `projectBearingRange` and `bearingRangeBetween`; `normalizeBearing` wraps degrees into $[0, 360)$:

```
742 function projectBearingRange(baseCoord, bearing, rangeFeet) {
743   if (!baseCoord || !Number.isFinite(bearing) || !Number.isFinite(rangeFeet)) return null;
744   const radians = normalizeBearing(bearing) * Math.PI / 180;
745   return {
746     northFeet: baseCoord.northFeet + Math.cos(radians) * rangeFeet,
747     eastFeet: baseCoord.eastFeet + Math.sin(radians) * rangeFeet,
748   };
749 }
750
751 function bearingRangeBetween(fromCoord, toCoord) {
752   if (!fromCoord || !toCoord) return null;
753   const deltaNorth = toCoord.northFeet - fromCoord.northFeet;
754   const deltaEast = toCoord.eastFeet - fromCoord.eastFeet;
755   const rangeFeet = Math.hypot(deltaNorth, deltaEast);
756   let bearing = Math.atan2(deltaEast, deltaNorth) * 180 / Math.PI;
757   if (bearing < 0) bearing += 360;
758   return {
759     brg: formatDraggedBearing(bearing),
760     rng: formatDraggedRange(rangeFeet),
761   };
762 }
```

The same file's formatters set the precision returned in `brg`/`rng` and used for dragged offsets:

```
923 function formatDraggedBearing(value) {
924   if (!Number.isFinite(value)) return "";
925   const rounded = Math.round(value * 10) / 10;
926   return Number.isInteger(rounded) ? String(rounded) : rounded.toFixed(1);
927 }
928
929 function formatDraggedRange(value) {
930   if (!Number.isFinite(value)) return "";
931   return String(Math.round(value));
932 }
```

6.2. **Magnetic headings.**

Values. Flight-plan `MHDG`, in whole degrees, and pop-up magnetic headings, to 0.1°. When variation is unavailable, the app identifies the headings as true (`THDG` or a °T suffix).

Formulas.

$$\theta_M = (\theta_T - \nu) \bmod 360^\circ,$$
$$\theta_{FP} = \lfloor \theta_M + 0.5^\circ \rfloor \bmod 360^\circ,$$
$$\theta_{POP} = \left(\frac{\lfloor 10\theta_M + 0.5^\circ \rfloor}{10} \right) \bmod 360^\circ.$$

Rounding operates on the numerical degree values. For a pilot-entered magnetic heading, the inverse conversion $\theta_T = (\theta_M + \nu) \bmod 360^\circ$ is applied before geometry.

Variables.

- θ_T, θ_M: true and magnetic heading, in degrees. $\theta_{FP}, \theta_{POP}$ are the rounded flight-plan and pop-up headings.
- ν: magnetic variation in degrees, positive east and negative west. It is bilinearly interpolated:

$$\nu = (1-a)(1-b)\nu_{00} + a(1-b)\nu_{10} + (1-a)b\nu_{01} + ab\nu_{11}.$$

- $x = N/c_k,\ y = E/c_k$: the magnetic grid's coordinates in km, where N, E are DTC north/east feet and $c_k = 1000/0.3048 = 3280.839895013123$ ft/km. Here the grid's x is *north*, and y is *east*.
- x_0, x_1 and y_0, y_1: adjacent grid-axis entries bracketing x, y. The fractional positions are $a = (x - x_0)/(x_1 - x_0)$ and $b = (y - y_0)/(y_1 - y_0)$.
- $\nu_{00}, \nu_{10}, \nu_{01}, \nu_{11}$: grid variations at (x_0, y_0), (x_1, y_0), (x_0, y_1), (x_1, y_1).

Data.

- θ_T for a flight-plan row is the heading column in `B/User/Briefings/briefing.txt`, parsed by `lib/parsers/parse_briefing_txt.py`. It is not recomputed from DTC positions. Pop-up headings come from the geometry described in section 7.
- N, E come from the matching numbered DTC steerpoint in `<callsign/save>.ini`. `lib/theater_paths.py:magnetic_variation_for_steerpoints` samples variation at each flight-plan row's point. The pop-up plan uses the selected reference steerpoint's variation throughout.
- x_0, x_1, y_0, y_1 and the four corner variations are read from the active terrain's `Weather/MagVarMap_*.CSV` by `openchart/vendor/opencam/theater.py:load_magnetic_variation_grid`. The header after `y\x` gives the x axis; the first entry of each subsequent row gives y; the remaining entries are signed degrees. The leading `#year` is retained as the grid epoch; no date-dependent correction is applied.
- ν is computed by `MagneticVariationGrid` in that file. A missing grid or an out-of-grid point leaves variation unavailable; the app then retains true headings.

Example. At STPT 2 of `I/Campaign/weight_tests.ini`, $(x, y) = (609.3402342, 514.3651638)$ km. `I/Terrdata/Israel/Weather/MagVarMap_Israel.CSV` brackets these by $(x_0, x_1) = (608, 624)$ and $(y_0, y_1) = (512, 528)$ km, with

$$(\nu_{00}, \nu_{10}, \nu_{01}, \nu_{11}) = (4.145995, 4.144043, 4.162748, 4.160932)^\circ.$$

Therefore $a = 0.08376464$, $b = 0.14782274$, and $\nu = 4.14830965^\circ$. The briefing's STPT 2 heading is 329°:

$$\theta_M = 329 - 4.14830965 = 324.85169035^\circ \quad \Longrightarrow \quad \texttt{MHDG 325}.$$

Code. `openchart/vendor/opencam/theater.py`, `MagneticVariationGrid`: `x_fraction`/`y_fraction` are a, b; `lower`/`upper` interpolate the two y rows. The campaign-coordinate wrapper shows the north/east axis order.

```
154  def variation_degrees(self, x_km: float, y_km: float) -> float:
155      """Bilinearly interpolate signed magnetic variation on the grid."""
156
157      x = _bounded_axis_value(x_km, self.x_km, "x_km")
158      y = _bounded_axis_value(y_km, self.y_km, "y_km")
159      x_index, x_fraction = _interpolation_position(x, self.x_km)
160      y_index, y_fraction = _interpolation_position(y, self.y_km)
161      lower_left = self.values_degrees[y_index][x_index]
162      lower_right = self.values_degrees[y_index][x_index + 1]
163      upper_left = self.values_degrees[y_index + 1][x_index]
164      upper_right = self.values_degrees[y_index + 1][x_index + 1]
165      lower = lower_left + (lower_right - lower_left) * x_fraction
166      upper = upper_left + (upper_right - upper_left) * x_fraction
167      return lower + (upper - lower) * y_fraction
168
169  def variation_at_campaign_position(
170      self,
171      position_x_ft: float,
172      position_y_ft: float,
173  ) -> float:
174      return self.variation_degrees(
175          position_x_ft / FEET_PER_KILOMETER,
176          position_y_ft / FEET_PER_KILOMETER,
177      )
```

`lib/html_gen.py`, `_flightplan_headings`, the branch where variation is available:

```
37      heading = str(math.floor((true_heading - variation) % 360 + 0.5) % 360)
38  headings[str(point.number)] = heading
```

`templates/dtc_popup.js`, `formatPopupHeading`; a missing variation contributes zero and is labelled true by `popupHeadingReference`:

```
12  function formatPopupHeading(trueHeading, plan) {
13    const heading = Number.parseFloat(trueHeading);
14    if (!Number.isFinite(heading)) return "---";
15    const magnetic = normalizeBearing(heading - (plan.magneticVariation ?? 0));
16    return formatPopupDegrees((Math.round(magnetic * 10) / 10) % 360);
17  }
```

The inverse conversion in the same file preserves true bearings for geometry and stored DTC offsets:

```
441  const magneticVariation = getPopupMagneticVariation();
442  // Convert the pilot's heading once; geometry and DTC offset rows stay true.
443  const trueHeading = normalizeBearing(heading + (magneticVariation ?? 0));
```

6.3. **Departure weather.**

Values. The ATIS table displays `WIND` (whole degrees/knots), `TEMP` (whole degrees Celsius), `QNH` (two decimals inHg and whole hPa), `WX` (category), `LOW CLD BASE` (whole ft MSL or `NONE`), and `FOG BOUNDARY` (km to one decimal).

Formulas.

$$QNH_{\text{inHg}} = \frac{p}{C_p}, \qquad \theta_W = \lfloor (\theta \bmod 360^\circ) + 0.5^\circ \rfloor \bmod 360^\circ.$$

Pressure is converted directly from the selected cell; the app does not apply an airport-elevation correction. Wind direction is wrapped and rounded without magnetic-variation correction. Other weather values are formatted from the stored cell.

Variables.

- p, QNH_{inHg}: cell pressure in hPa and its inHg equivalent. $C_p = 33.8638866667$ hPa/inHg is the application's fixed conversion constant, `HPA_PER_INHG`.
- θ, θ_W, U: stored first-layer wind direction, rounded displayed direction (degrees), and first-layer speed (knots).
- T, κ, H_c, F: cell temperature (°C), weather-type code, low-cloud base (ft MSL), and stored fog-end distance (km). F is shown as a fog boundary, not a computed visibility.
- N, E: departure north/east coordinates in ft; $N_k = 0.0003048N$, $E_k = 0.0003048E$ convert them to km.
- L: theater side length in km; w, h: FMAP width and height in cells.
- c, r: zero-based column and row of the sampled cell:

$$c = \lfloor wE_k/L - 0.5 \rfloor \bmod w, \qquad r = \lceil h(1 - N_k/L) - 0.5 \rceil \bmod h.$$

  Columns run eastward and rows southward. The fixed half-cell offset and edge wrapping implement the stored-cell convention used by `_departure_cell`; coordinates must be within $0 \le N_k, E_k \le L$.

Data.

- N, E are read from the unique steerpoint with action 1 in `[STPT]` of the loaded DTC `<callsign/save>.ini`. L comes from the active theater's `Theater size in KM` in `NewTerrain/Theater.txt`, through `lib/theater_paths.py:read_theater_map_info`.
- $w, h, p, T, \theta, U, \kappa, H_c, F$ come from the loaded CAM's same-named adjacent `<save>.fmap`, parsed by `lib/cam/opencam/fmap_parser.py` and `lib/cam/opencam/fmap_wrappers.py`. The selected cell has array index $rw+c$. Its fields are `pressure_hpa`, `temperature_c`, `wind_directions_deg[0]`, `wind_speeds_knots[0]`, `weather_type`, `cumulus_base_ft` and `fog_end_km`. No spatial interpolation or movement in time is applied to the saved snapshot.
- κ is mapped in `lib/atis.py` as 1 sunny, 2 fair, 3 poor, 4 inclement. Sunny displays `NONE` for low cloud; types 2–4 display a valid H_c. Unrecognised codes display `UNKNOWN (code)`.
- $QNH_{\text{inHg}}, \theta_W$ and the formatted cell values are computed by `lib/atis.py:build_departure_atis` and displayed by `templates/atis.html`. Missing or invalid fields remain unavailable; a missing/mismatched FMAP produces a status message.

Example. `B/Data/Campaign/714_html_test.fmap` has $w = h = 59$. For the departure in `test/User/Config/wsy.ini`, $(N, E) = (1700669.625, 2468185)$ ft, and Korea's `B/Data/TerrData/Korea/NewTerrain/Theater.txt` gives $L = 1024$ km:

$$\begin{aligned}(N_k, E_k) &= (518.3641017, 752.302788) \text{ km},\\ c &= \lfloor 59(752.302788)/1024 - 0.5 \rfloor \bmod 59 = 42,\\ r &= \lceil 59(1 - 518.3641017/1024) - 0.5 \rceil \bmod 59 = 29.\end{aligned}$$

Cell $29 \cdot 59 + 42 = 1753$ has $p = 1011.96533203$ hPa, $\theta = 39.63571167°$, $U = 9.66757011$ kt, $T = 5.97265148$°C, $\kappa = 3$, $H_c = 2159.93506$ ft and $F = 59.71747$ km. In particular,

$$QNH_{\text{inHg}} = 1011.96533203/33.8638866667 = 29.88332 \longrightarrow 29.88.$$

The readout is `WIND: 040 DEG / 10 KT`, `TEMP: 6 DEG C`, `QNH: 29.88 INHG / 1012 HPA`, `WX: POOR`, `LOW CLD BASE: 2,160 FT MSL`, `FOG BOUNDARY: 59.7 KM`.

Code. `lib/atis.py`, `_departure_cell`, coordinate conversion and cell selection:

```
north_km, east_km = north_ft * 0.0003048, east_ft * 0.0003048
if not (0 <= north_km <= theater_size_km and 0 <= east_km <= theater_size_km):
    raise ValueError("Departure is outside the active theater")
column = math.floor(east_km / theater_size_km * weather.width - 0.5) % weather.width
row = math.ceil((1 - north_km / theater_size_km) * weather.height - 0.5) % weather.height
return weather.cell(row, column)
```

The same file's `build_departure_atis` converts and formats the fields from that cell; `_finite` rejects non-finite values:

```
result["weather_type"] = WEATHER_TYPES.get(cell.weather_type, f"Unknown ({cell.weather_type})")
# The per-cell cloud-base field controls the low cloud layer (ft MSL).
# Sunny disables that layer even when the FMAP retains a base value.
cloud_base = _finite(cell.cumulus_base_ft)
if cell.weather_type == 1:
    result["cloud_base"] = "None"
elif cell.weather_type in (2, 3, 4) and cloud_base is not None and cloud_base >= 0:
    result["cloud_base"] = f"{cloud_base:,.0f} ft MSL"
# Preserve the stored fog-end distance. Live ATIS visibility has not
# been established as a direct conversion of this input to BMS's model.
fog_boundary = _finite(cell.fog_end_km)
if fog_boundary is not None and fog_boundary >= 0:
    result["fog_boundary"] = f"{fog_boundary:.1f} km"

pressure, temperature = _finite(cell.pressure_hpa), _finite(cell.temperature_c)
speed, direction = _finite(cell.wind_speeds_knots[0]), _finite(cell.wind_directions_deg[0])
if speed is not None and speed >= 0 and direction is not None:
    result["wind"] = f"{math.floor(direction % 360 + 0.5) % 360:03d} deg / {speed:.0f} kt"
if temperature is not None:
    result["temperature"] = f"{temperature:.0f} deg C"
if pressure is not None and pressure > 0:
    result["qnh"] = f"{pressure / HPA_PER_INHG:.2f} inHg / {pressure:.0f} hPa"
```

6.4. **Terrain and AGL-to-MSL conversion.**

Values. The pop-up calculator displays AGL and MSL altitudes in whole feet for `PDWN`, `APEX`, `RALT` and `MRA`; map labels also use this conversion.

Formula.

$$H_{\mathrm{MSL}} = H_{\mathrm{AGL}} + H_{\mathrm{terrain}}.$$

Variables.

- $H_{\mathrm{AGL}}, H_{\mathrm{MSL}}$: height above ground and altitude above mean sea level, in ft.
- H_{terrain}: signed ground elevation in ft, read from one heightmap sample at the requested position.
- N, E: requested north/east coordinates in ft. $N_{\max} = E_{\max} = 1000L/0.3048$ are the theater extents in ft, where L is the theater side length in km.
- w, h: heightmap width and height in pixels. x, y: zero-based pixel column and row, computed as

  $$x = \mathrm{clip}(\mathrm{trunc}[(w-1)E/E_{\max}], 0, w-1),$$
  $$y = \mathrm{clip}(\mathrm{trunc}[(h-1)(1-N/N_{\max})], 0, h-1).$$

  trunc truncates to an integer; clip clamps to the pixel bounds. Columns run eastward and rows southward. Out-of-theater positions are rejected before this calculation.
- $o = 2(yw + x)$: byte offset of the sample in the RAW file. The factor 2 is the byte size of one signed 16-bit elevation.

Data.

- H_{AGL}, N, E come from the pop-up plan in `templates/dtc_popup.js`; its inputs and geometry are described in section 7. `PDWN`/`APEX` use terrain at the pull-down point; `RALT`/`MRA` use terrain at the release point (or target if that point is absent).
- L, w, h are read from `Theater size in KM` and `Map size in pixels` in the active `NewTerrain/Theater.txt`, resolved through the theater TDF's `terraindir`. `lib/terrain_elevation.py:_resolve_heightmap_spec` sets both extents from L and both dimensions from the pixel size. If the pixel size is absent, it infers a square side from half the RAW byte count, falling back to 32768 if that is not a square.

- H_{terrain} is read from the active `NewTerrain/Heightmaps/Heightmap.RAW` by `lib/terrain_elevation.py: HeightmapSampler.elevation_feet`. The two bytes at o are decoded as a little-endian signed 16-bit integer in feet. This is a single-pixel lookup without interpolation. The heightmap lookup approach is also credited to oakdesign's converter [5] in `README.md`; see its `CoordinateConverter.get_elevation`.
- H_{MSL} is computed by `popupAglMslLine` and related formatters in `templates/dtc_popup.js`. Until a finite terrain sample is available, the label reads `MSL pending`.

Example. `I/Terrdata/Israel/NewTerrain/Theater.txt` gives $L = 1024$ km and $w = h = 32768$, hence $N_{\max} = E_{\max} = 3359580.05249$ ft. At STPT 1 of `I/Campaign/weight_tests.ini`, $(N, E) = (1923708.25, 1733469.375)$ ft:

$$x = 16907, \qquad y = 14004, \qquad o = 2(14004 \cdot 32768 + 16907) = 917799958.$$

The sample in `I/Terrdata/Israel/NewTerrain/Heightmaps/Heightmap.RAW` is $H_{\text{terrain}} = 154$ ft. An illustrative $H_{\text{AGL}} = 2000$ ft at that position would display $H_{\text{MSL}} = 2000 + 154 = 2154$ ft. The airport elevation in the DTC is instead $-Z = 120$ ft; section 5 uses that DTC value.

Code. `lib/terrain_elevation.py`, `HeightmapSampler.elevation_feet`, after the theater-bound check: map to a pixel, seek to its two bytes, and decode the signed elevation.

```
east_ratio = east_feet / self.spec.max_east_feet if self.spec.max_east_feet else 0
north_ratio = north_feet / self.spec.max_north_feet if self.spec.max_north_feet else 0
pixel_x = int(east_ratio * (self.spec.width - 1))
pixel_y = int((1.0 - north_ratio) * (self.spec.height - 1))
pixel_x = max(0, min(self.spec.width - 1, pixel_x))
pixel_y = max(0, min(self.spec.height - 1, pixel_y))

offset = (pixel_y * self.spec.width + pixel_x) * 2
with self._lock:
    if self._file is None:
        try:
            self._file = self.spec.path.open("rb")
        except OSError as exc:
            raise HeightmapReadError(f"Could not open heightmap {self.spec.path}: {exc}") from exc
    try:
        self._file.seek(offset)
        raw = self._file.read(2)
    except OSError as exc:
        raise HeightmapReadError(f"Could not read heightmap {self.spec.path}: {exc}") from exc

if len(raw) != 2:
    raise HeightmapReadError(f"Expected 2 bytes at offset {offset}, got {len(raw)}")
return int(struct.unpack("<h", raw)[0])
```

`templates/dtc_popup.js`, `popupAglMslLine`; `aglFeet` and `terrainFeet` are H_{AGL} and H_{terrain}:

```
function popupAglMslLine(label, aglFeet, terrainFeet) {
  const aglText = formatPopupNumber(aglFeet, 0) + " ft";
  const mslText = Number.isFinite(terrainFeet)
    ? formatPopupNumber(aglFeet + terrainFeet, 0) + " ft"
    : "MSL pending";
  return popupLine(label, popupMetric("AGL", aglText), popupMetric("MSL", mslText));
}
```

7. Pop-up attack calculator

The calculator uses pilot-entered attack parameters and fixed application constants. BMS data supplies the reference positions, magnetic variation and terrain elevations; the model does not read weapon or aircraft aerodynamic tables.

7.1. **Bomb range.**

Value. `BOMB RANGE` in the DTC pop-up inputs and support values, in whole feet. Low/high-drag modes compute it; manual mode uses the entered range.

Formulas. For each time step before ground crossing:

$$a_j = \begin{cases} -kA_d|u_j|/\sqrt{u_j^2+\epsilon}, & u_j > 0, \\ 0, & u_j = 0, \end{cases}$$

$$\Delta x_j = u_j\Delta t + \tfrac{1}{2}a_j\Delta t^2, \qquad \Delta h_j = -w_j\Delta t - \tfrac{1}{2}g\Delta t^2,$$
$$x_{j+1} = x_j + \Delta x_j, \qquad h_{j+1} = h_j + \Delta h_j,$$
$$u_{j+1} = \max(0, u_j + a_j\Delta t), \qquad w_{j+1} = w_j + g\Delta t.$$

The final step is corrected by its overshoot fraction:

$$B = x_m - f\Delta x_{m-1}, \qquad t_f = (m-f)\Delta t.$$

Variables.

- H, V, δ: release height in ft AGL, release TAS in knots, and positive dive angle in degrees.
- x_j, h_j, u_j, w_j: accumulated horizontal range, remaining height (ft), horizontal speed and downward speed (ft/s) at step j. Initially $x_0 = 0$, $h_0 = H$, and

  $$u_0 = \text{round}_{0.01}(c_V V\cos\delta), \qquad w_0 = \text{round}_{0.01}(c_V V\sin\delta).$$

  $\text{round}_{0.01}$ rounds to 0.01 ft/s; $c_V = 1.68781$ ft/s per knot is `FEET_PER_KNOT_SECOND`.
- $\Delta t = 0.1$ s: fixed time step. $j = 0, \ldots, m-1$ is the step index; m is the first index for which $h_m \le 0$.
- a_j: horizontal acceleration in ft/s^2; $\Delta x_j, \Delta h_j$: range and signed height changes during step j.
- k, g: fixed model parameters, $(0.165, 32.177)$ for low drag and $(1, 20.91505)$ for high drag. k is dimensionless and g is the model's downward acceleration in ft/s^2; the high-drag value is an effective acceleration. $A_d = 140$ ft/s^2 and $\epsilon = 0.1$ (ft/s)^2 are fixed constants in the horizontal-drag expression.
- $f = h_m/\Delta h_{m-1}$: fraction of the final step below ground. Both numerator and denominator are nonpositive. B is the corrected range in ft and t_f the corrected fall time in seconds. The latter feeds `TOF STICK` in section 7.2.

Data.

- H, V, δ come from the `data-popup-field` controls `releaseAltitude`, `releaseSpeed` and `diveAngle` in `templates/dtc.html`. The `rangeModel` control selects `low_drag`, `high_drag` or `manual`. These are pilot inputs, not values extracted from a BMS weapon file.
- c_V is declared in `templates/dtc.html`. $\Delta t, k, g, A_d, \epsilon$ are literals in `templates/dtc_popup.js`: `computeDragBombTrajectory`. The approximation has no wind or atmospheric-density input.
- B, t_f are returned by `computeDragBombTrajectory`. `computePopupPlan` retains the unrounded range for geometry and writes a whole-foot value into the range input. Manual mode reads B from `bombRange` and leaves t_f unavailable. A failed trajectory or nonpositive range rejects the plan.

Example. The input fixture in `tests/popup_briefing.test.cjs` uses $H = 2000$ ft, $V = 520$ kt, $\delta = 15°$ and low drag: $u_0 = 847.76$, $w_0 = 227.16$ ft/s. The first step gives $a_0 = -23.10000$ ft/s^2, $x_1 = 84.66050$ ft and $h_1 = 1977.12312$ ft. Ground crossing occurs at $m = 62$:

$$h_{62} = -26.83394, \quad \Delta h_{61} = -42.504855, \quad x_{62} = 4812.13003, \quad \Delta x_{61} = 70.56950 \text{ ft}.$$

Thus $f = 0.63131471$, $B = 4767.57847$ ft and $t_f = 6.13687$ s; the displayed range is `4768`. The fixture's manual range 5138 ft is unused in this mode.

Code. `templates/dtc_popup.js`, `computeDragBombTrajectory`. The guard rejects a trajectory reaching 10000 iterations. `altitude`/`lastStepAltitudeChange` gives f:

```
299 function computeDragBombTrajectory(releaseAltitude, releaseSpeed, diveAngle, rangeModel) {
300   const stepSeconds = 0.1;
301   const dragScale = rangeModel === "high_drag" ? 1.0 : 0.165;
302   const gravity = rangeModel === "high_drag" ? 20.91505 : 32.177;
303   const diveRadians = diveAngle * Math.PI / 180;
304   let horizontalSpeed = Math.round(Math.cos(diveRadians) * releaseSpeed * FEET_PER_KNOT_SECOND * 100) / 100;
305   let verticalSpeedDown = Math.round(Math.sin(diveRadians) * releaseSpeed * FEET_PER_KNOT_SECOND * 100) / 100;
306   let rangeFeet = 0;
307   let altitude = releaseAltitude;
308   let lastStepRange = 0;
309   let lastStepAltitudeChange = 0;
310   let timeSeconds = 0;
311   let guard = 0;
312   while (altitude > 0 && guard < 10000) {
313     guard += 1;
```

```
314     const horizontalDrag = dragScale * 140 * Math.abs(horizontalSpeed) / Math.sqrt(horizontalSpeed *
        horizontalSpeed + 0.1);
315     const acceleration = horizontalSpeed > 0 ? -horizontalDrag : 0;
316     lastStepRange = horizontalSpeed * stepSeconds + 0.5 * acceleration * stepSeconds * stepSeconds;
317     lastStepAltitudeChange = -verticalSpeedDown * stepSeconds - 0.5 * gravity * stepSeconds * stepSeconds;
318     rangeFeet += lastStepRange;
319     altitude += lastStepAltitudeChange;
320     timeSeconds += stepSeconds;
321     horizontalSpeed = Math.max(0, horizontalSpeed + acceleration * stepSeconds);
322     verticalSpeedDown += gravity * stepSeconds;
323   }
324   if (guard >= 10000 || !Number.isFinite(altitude) || !Number.isFinite(lastStepAltitudeChange) ||
        lastStepAltitudeChange === 0) return null;
325   const overshootFraction = altitude / lastStepAltitudeChange;
326   return {
327     rangeFeet: rangeFeet - overshootFraction * lastStepRange,
328     timeSeconds: timeSeconds - overshootFraction * stepSeconds,
329   };
330 }
```

7.2. **Stick, tracking and climb profile.**

Values. The calculator and Pop-up briefing display `TOF STICK` (seconds to one decimal), `CA` (climb angle), `PDWN/APEX` (altitudes), and `AOD`, `PUP`, `RADIUS`, `MAP` (distances in whole feet). Support values include `STICK LEN`, `STICK DUR`, `RELEASE RANGE`, `GROUND SPEED`, `TRACKING`, `TRACK ALT` and `ANGLE OFF`. AGL heights are also shown in MSL using section 6.4.

Formulas.

$$V_h = V\cos\delta, \qquad L = (n-1)s, \qquad t_s = \frac{L}{c_V V_h},$$

$$B_r = B + \begin{cases} L/2, & \text{MID}, \\ 0, & \text{START}, \end{cases} \qquad t_{\text{stick}} = t_f + t_s,$$

$$X_{\text{track}} = c_G V_h t, \qquad Z_{\text{track}} = c_G V t \sin\delta, \qquad H_{\text{track}} = H + Z_{\text{track}},$$

$$MAP = B_r + X_{\text{track}}, \qquad AOD = \frac{H}{\tan\delta} - B_r,$$

$$\chi = \begin{cases} \delta + 5^\circ, & \delta \le 15^\circ, \\ \delta + 10^\circ, & \delta > 15^\circ, \end{cases} \qquad \Omega = 2\chi, \qquad R = \frac{(c_G V)^2}{g_T G},$$

$$H_{\text{PD}} = H_{\text{track}} - R(\cos\delta - \cos\chi), \qquad H_{\text{apex}} = H_{\text{PD}} + R(1-\cos\chi),$$

$$d_{\text{pop}} = \frac{H_{\text{PD}} - H_{\text{in}}}{\tan\chi}.$$

Variables.

- H, V, δ: release height (ft AGL), TAS (knots) and dive angle, as in section 7.1. B, t_f are its bomb range (ft) and fall time (s).
- V_h: horizontal component of release TAS, in knots; the app labels it `GROUND SPEED`. Wind is not included.
- $n = \max(1, \lfloor n_{\text{input}} + 0.5 \rfloor)$: integer pulse count, from the entered n_{input}. s is spacing in ft; L is total stick length in ft and t_s is ripple duration in seconds.
- B_r: target-to-first-release distance in ft. MID adds half the stick length to centre the stick on the target; START uses B. t_{stick} is time from first release to final impact, in seconds.
- t: entered tracking time in seconds. $X_{\text{track}}, Z_{\text{track}}$: horizontal distance and height lost during tracking, in ft; H_{track}: tracking-start height in ft AGL.
- MAP: target-to-maneuver-aim-point distance in ft; AOD: aim-off distance beyond the target, in ft.
- χ, Ω: climb angle and ingress/attack angle-off, in degrees. G is the selected dimensionless G value: 3.5 or 5. R is the turn/pull-down radius in ft.
- $H_{\text{PD}}, H_{\text{apex}}, H_{\text{in}}$: pull-down, apex and entered ingress heights in ft AGL. d_{pop} is the horizontal PUP-to-pull-down distance in ft, displayed as `PUP`.
- $c_V = 1.68781$ and $c_G = 1.69$: application conversion factors in ft/s per knot, used for ripple timing and tracking/turn geometry respectively. $g_T = 32.2$ ft/s^2 is the fixed acceleration constant in the radius formula.

Data.

- $H, V, \delta, H_{\text{in}}, t, n_{\text{input}}, s$ come from `releaseAltitude`, `releaseSpeed`, `diveAngle`, `ingressAltitude`, `timeOnFinal`, `ripplePulses` and `stickSpacing` in `templates/dtc.html`. `stickMode` selects MID/START.
- G is selected by `gProfile`: the UI labels `3-3.5 G` and `4.5-5 G` correspond to fixed values 3.5 and 5 in `templates/dtc_popup.js:computePopupPlan`. The same function contains c_G, g_T and the climb-angle rule; c_V is declared in `templates/dtc.html`.
- B, t_f come from section 7.1; manual range mode leaves t_{stick} unavailable, displayed as `N/A`. The remaining values are computed in `computePopupPlan` and formatted by `formatPopupOutput` and `formatPopupBriefing` in `templates/dtc_popup.js`. The app rejects $AOD \le 0$ or $d_{\text{pop}} \le 0$.

Example. Continuing the fixture in `tests/popup_briefing.test.cjs`, $n = 2$, $s = 100$ ft, MID, $t = 5$ s, $H_{\text{in}} = 500$ ft and $G = 3.5$:

$$V_h = 520\cos 15^\circ = 502.28143 \text{ kt}, \qquad L = (2-1)100 = 100 \text{ ft},$$
$$t_s = \frac{100}{1.68781(502.28143)} = 0.11796 \text{ s}, \qquad B_r = 4767.57847 + 50 = 4817.57847 \text{ ft},$$
$$t_{\text{stick}} = 6.13687 + 0.11796 = 6.25483 \text{ s}, \qquad MAP = 4817.57847 + 4244.27808 = 9061.85655 \text{ ft}.$$

The same calculation gives $AOD = 2646.52314$ ft, $\chi = 20^\circ$, $\Omega = 40^\circ$, $R = 6852.61260$ ft, $H_{\text{track}} = 3137.25088$ ft, $H_{\text{PD}} = 2957.48489$ ft, $H_{\text{apex}} = 3370.74800$ ft and $d_{\text{pop}} = 6751.88424$ ft. The briefing shows `TOF STICK 6.3 sec`, `CA 20 deg`, `PDWN 2957`, `APEX 3371`, `AOD 2647`, `PUP 6752`, `RADIUS 6853`, `MAP 9062` (feet). Terrain is absent in this fixture, so MSL labels remain pending.

Code. `templates/dtc_popup.js`, `computePopupPlan`; `ripplePulses` has already been rounded and clamped to at least 1:

```
406  const diveRadians = diveAngle * Math.PI / 180;
407  const groundSpeed = releaseSpeed * Math.cos(diveRadians);
408  const groundSpeedFps = groundSpeed * FEET_PER_KNOT_SECOND;
409  const stickLength = Math.max(0, ripplePulses - 1) * stickSpacing;
410  const stickHalfLength = stickMode === "mid" ? stickLength / 2 : 0;
411  const stickDuration = groundSpeedFps > 0 ? stickLength / groundSpeedFps : NaN;
412  const releaseReferenceRange = bombRange + stickHalfLength;
413  const stickTimeOfFall = Number.isFinite(bombTimeOfFall) && Number.isFinite(stickDuration)
414    ? bombTimeOfFall + stickDuration
415    : NaN;
416  const horizontalTrackingDistance = groundSpeed * 1.69 * timeOnFinal;
417  const verticalTrackingDistance = releaseSpeed * 1.69 * timeOnFinal * Math.sin(diveRadians);
418  const mapDistance = releaseReferenceRange + horizontalTrackingDistance;
419  const actionRangeFeet = actionRangeNm * FEET_PER_NM;
420  const aod = releaseAltitude / Math.tan(diveRadians) - releaseReferenceRange;
421  if (aod <= 0) return { error: "AOD is negative for this profile. Increase release altitude or reduce bomb range."
         };
422  const climbAngle = diveAngle <= 15 ? diveAngle + 5 : diveAngle + 10;
423  const climbRadians = climbAngle * Math.PI / 180;
424  const angleOff = 2 * climbAngle;
425  const turnG = gProfile === "high" ? 5 : 3.5;
426  const trackAltitude = releaseAltitude + verticalTrackingDistance;
427  const turnRadius = ((releaseSpeed * 1.69) ** 2) / (32.2 * turnG);
```

Later in the same function, the tracking height and radius give the pull-down/apex heights and PUP-to-pull-down distance:

```
436  const pullDownAltitude = trackAltitude - turnRadius * (Math.cos(diveRadians) - Math.cos(climbRadians));
437  const apexAltitude = pullDownAltitude + turnRadius * (1 - Math.cos(climbRadians));
438  const popToPullDownDistance = (pullDownAltitude - ingressAltitude) / Math.tan(climbRadians);
439  if (popToPullDownDistance <= 0) return { error: "Ingress altitude is above the computed pull-down altitude." };
```

7.3. **Recovery height and clearance.**

Values. `ALTLOSS`, `GNDC` and `MRA`, in whole feet. `GNDC` is AGL; `MRA` is shown in AGL and MSL. Support values show `DELAY LOSS`, `PULL LOSS`, `PULL RADIUS`, `MRA BUFFER`, and the recovery G/delay.

Formulas.

$$t_d = t_s + \tau_d, \qquad R_r = \frac{(c_V V)^2}{g_T G_r},$$
$$\Delta H_d = c_V V \sin\delta\, t_d, \qquad \Delta H_r = R_r(1-\cos\delta),$$
$$\Delta H = \Delta H_d + \Delta H_r, \qquad H_{\text{clear}} = H - \Delta H, \qquad H_{\text{MRA}} = \Delta H + H_b.$$

Variables.

- H, V, δ, t_s: release height (ft AGL), release TAS (knots), dive angle and ripple duration (s), from section 7.2.
- $\tau_d = 2$ s: fixed delay after the last release; t_d: total delay from the first release to the recovery pull.
- $G_r = 5$: fixed recovery G value, independent of the selected turn profile. R_r is the recovery pull radius in ft. $c_V = 1.68781$ ft/s per knot and $g_T = 32.2$ ft/s^2 retain the application constants above.
- $\Delta H_d, \Delta H_r, \Delta H$: height lost during the delay, during the pull to level, and their sum, in ft.
- H_{clear}: remaining height above ground after recovery, in ft; H_{MRA}: release height needed by this model to leave the buffer $H_b = 120$ ft. MSL conversion follows section 6.4.

Data.

- H, V, δ come from the same DTC pop-up input controls as above; t_s is computed from pulse count and spacing in `templates/dtc_popup.js:computePopupPlan`.
- τ_d, G_r, g_T are fixed literals in `computePopupPlan`. H_b is `POPUP_MRA_BUFFER_FEET`, and c_V is `FEET_PER_KNOT_SECOND`, both declared in `templates/dtc.html`. These parameters are not selected from aircraft-specific recovery tables.
- $\Delta H, H_{\text{clear}}, H_{\text{MRA}}$ are computed by `computePopupPlan`; `formatPopupOutput` and `formatPopupBriefing` display them. The arithmetic assumes a common ground reference through the recovery; it does not sample terrain along a recovery path.

Example. For the same `tests/popup_briefing.test.cjs` fixture:

$$t_d = 0.11795852 + 2 = 2.11795852 \text{ s},$$
$$R_r = \frac{(1.68781 \cdot 520)^2}{32.2 \cdot 5} = 4784.40486 \text{ ft},$$
$$\Delta H_d = 1.68781 \cdot 520 \sin 15^\circ \cdot 2.11795852 = 481.10579 \text{ ft},$$
$$\Delta H_r = 4784.40486(1-\cos 15^\circ) = 163.02464 \text{ ft}.$$

Hence $\Delta H = 644.13043$ ft, $H_{\text{clear}} = 2000 - 644.13043 = 1355.86957$ ft and $H_{\text{MRA}} = 644.13043 + 120 = 764.13043$ ft. The briefing displays `ALTLOSS 644 ft`, `GNDC 1356 ft AGL`, and `MRA 764 ft AGL` (with MSL pending in this fixture).

Code. `templates/dtc_popup.js`, `computePopupPlan`, delay and pull contributions:

```
428  const recoveryPullDelay = stickDuration + 2;
429  const recoveryPullG = 5;
430  const releaseSpeedFps = releaseSpeed * FEET_PER_KNOT_SECOND;
431  const recoveryDelayAltitudeLoss = releaseSpeedFps * Math.sin(diveRadians) * recoveryPullDelay;
432  const recoveryPullRadius = (releaseSpeedFps ** 2) / (32.2 * recoveryPullG);
433  const recoveryPullAltitudeLoss = recoveryPullRadius * (1 - Math.cos(diveRadians));
434  const recoveryAltitudeLoss = recoveryDelayAltitudeLoss + recoveryPullAltitudeLoss;
435  const recoveryGroundClearance = releaseAltitude - recoveryAltitudeLoss;
```

The same function adds the buffer when returning the plan values:

```
533  recoveryMraAltitude: recoveryAltitudeLoss + POPUP_MRA_BUFFER_FEET,
534  recoveryMraBuffer: POPUP_MRA_BUFFER_FEET,
```

7.4. **Map points, turn and navigation offsets.**

Values. The map shows action, PUP, pull-down, MAP, target and OA1 points and the connecting turn. `POP BRG/POP RNG` give the PUP-to-target bearing and range (degrees to one decimal, NM to one decimal). Support values show point-to-point distances, headings, and `PUP OFFSET/OA1 OFFSET` bearing/range pairs (true degrees to one decimal and whole feet). The action-point tooltip also shows the `Offset` angle between ingress and the leg to PDP.

Formulas. The entered heading determines either θ_A or θ_I; the other follows from the selected side:

$$\theta_I = (\theta_A - \eta\Omega) \bmod 360^\circ, \qquad \theta_A = (\theta_I + \eta\Omega) \bmod 360^\circ.$$

Construct the action point, MAP, release point and aim-off point:

$$\mathbf{P} = \mathbf{T} - d_A\mathbf{e}(\theta_I), \qquad \mathbf{M} = \mathbf{T} - MAP\,\mathbf{e}(\theta_A),$$
$$\mathbf{X} = \mathbf{T} - B_r\mathbf{e}(\theta_A), \qquad \mathbf{O} = \mathbf{T} + AOD\,\mathbf{e}(\theta_A).$$

For the circle tangent to the attack heading at MAP:

$$\mathbf{C} = \mathbf{M} + \eta R\mathbf{n}(\theta_A), \qquad \mathbf{v} = \mathbf{P} - \mathbf{C}, \qquad \mathbf{Q}_\pm = \mathbf{C} + \frac{R^2}{\|\mathbf{v}\|^2}\mathbf{v} \pm \frac{R\sqrt{\|\mathbf{v}\|^2 - R^2}}{\|\mathbf{v}\|^2}J\mathbf{v}.$$

The selected pull-down point $\mathbf{Q}$ is the candidate whose computed arc endpoint is closest to $\mathbf{M}$. Then

$$\mathbf{U} = \mathbf{Q} - d_{\text{pop}}\mathbf{e}(\theta_o), \qquad D_{\text{POP}} = \frac{\|\mathbf{T} - \mathbf{U}\|}{c_N}, \qquad \omega = \min(\alpha, 360^\circ - \alpha).$$

Bearings and the remaining straight-line distances use the formulas in section 6.1; magnetic display uses section 6.2.

Variables.

- All bold point vectors are (N, E) in feet. $\mathbf{S}, \mathbf{T}$: selected reference steerpoint and target. $\mathbf{P}, \mathbf{M}, \mathbf{X}, \mathbf{O}, \mathbf{Q}, \mathbf{U}$: action, MAP, first release, OA1, pull-down (PDP), and PUP positions.
- $\theta_A, \theta_I, \theta_o$: true attack, ingress and action-to-PDP headings, in degrees. $\eta = +1$ for a right turn and -1 for a left turn. Ω is the angle-off from section 7.2.
- $\mathbf{e}(\theta) = (\cos\theta, \sin\theta)$: forward unit vector; $\mathbf{n}(\theta) = (-\sin\theta, \cos\theta)$: right unit normal. The arguments are converted to radians for evaluation.
- $d_A = c_N r_A$: action-to-target distance in ft, from entered range r_A in NM; $c_N = 6076.12$ ft/NM is `FEET_PER_NM`. $MAP, B_r, AOD, R, d_{\text{pop}}$ are the distances/radius from section 7.2, in ft.
- $\mathbf{C}$: turn-circle centre at MAP; $\mathbf{v}$: vector from that centre to the action point. $J(v_N, v_E) = (v_E, -v_N)$ is a quarter-turn operator. $\|\cdot\|$ is Euclidean length. The two tangent candidates $\mathbf{Q}_+, \mathbf{Q}_-$ require $\|\mathbf{v}\| > R$.
- $j \in \{+, -\}$: tangent-candidate index. For each candidate, θ_j is the true bearing from $\mathbf{P}$ to $\mathbf{Q}_j$, rounded to 0.1° by `bearingRangeBetween`. The arc is reconstructed from that rounded heading:

$$\mathbf{C}_j = \mathbf{Q}_j + \eta R\mathbf{n}(\theta_j),$$
$$\psi_j = \eta\big[(\eta(\theta_A - \theta_j)) \bmod 360^\circ\big],$$
$$\mathbf{A}_j = \mathbf{C}_j + \mathcal{R}_{\psi_j}(\mathbf{Q}_j - \mathbf{C}_j).$$

  $\mathbf{C}_j$ is the reconstructed centre, ψ_j the signed turn angle, and $\mathbf{A}_j$ the arc endpoint. The rotation is $\mathcal{R}_\psi(N, E) = (N\cos\psi - E\sin\psi,\ N\sin\psi + E\cos\psi)$. For $|\psi_j| < 0.1^\circ$, the code returns $\mathbf{A}_j = \mathbf{Q}_j$.
- $j_* = \arg\min_{j\in\{+,-\}} \|\mathbf{A}_j - \mathbf{M}\|$: chosen candidate; $\mathbf{Q} = \mathbf{Q}_{j_*}$ and $\theta_o = \theta_{j_*}$. This is an endpoint-error comparison; the code applies no maximum-error threshold.
- $\alpha = (\theta_o - \theta_I) \bmod 360^\circ$: wrapped difference between the offset-leg and ingress headings. ω is the smaller angle between them, displayed as `Offset`.
- D_{POP}: PUP-to-target range in NM. The PUP and OA1 offset rows are measured from $\mathbf{S}$ to $\mathbf{U}$ and $\mathbf{O}$, respectively.

Data.

- $\mathbf{S}$ is read from the selected `[STPT].target_k` in the loaded `<callsign/save>.ini`, through `getBmsCoordForStpt` in `templates/dtc.html`. VRP mode uses $\mathbf{T} = \mathbf{S}$. VIP mode projects $\mathbf{T}$ from $\mathbf{S}$ using the `VIP` bearing/range in `[NAV OFFSETS]` or its edited DTC fields, as in section 6.1.
- r_A, η and the entered heading come from `actionRangeNm`, `side` and `heading`; `headingMode` selects attack or ingress. These are controls in `templates/dtc.html`. The entered heading is converted to true using the selected point's variation as described in section 6.2. c_N is declared in `templates/dtc.html`.
- $\mathbf{C}, \mathbf{Q}_\pm, \theta_j, \mathbf{A}_j, j_*$ are computed by `getTurnCenterAtFinalPoint`, `getCircleTangentsFromPoint`, `bearingRangeBetween`, `buildTurnArcCoords` and `solvePopupTurnGeometry` in `templates/dtc.html`. The chosen tangent need not land exactly at MAP after heading rounding.
- $\mathbf{P}, \mathbf{M}, \mathbf{X}, \mathbf{O}, \mathbf{U}$ and their bearing/range pairs are computed by `templates/dtc_popup.js:computePopupPlan`. Missing target data, no tangent, or $d_{\text{pop}} > \|\mathbf{Q} - \mathbf{P}\|$ rejects the plan. `applyPopupPlan` writes the computed offsets into the selected `OA1` row and `VIPPUP`/`VRPPUP`.
- D_{POP} and the displayed point distances use unrounded coordinates; bearings first pass through the 0.1° DTC formatter. The support offset rows retain true bearings, while the pop-up heading labels apply the magnetic conversion.

Example. The same `tests/popup_briefing.test.cjs` fixture supplies the synthetic reference $\mathbf{S} = (100000, 100000)$ ft, VRP mode, $r_A = 4.5$ NM, a right turn and attack heading 130° magnetic. Its mocked variation is $-8°$, giving $\theta_A = 122°$, $\theta_I = 82°$ and $d_A = 27342.54$ ft. Together with the profile above:

$$\mathbf{P} = (96194.65, 72923.56), \qquad \mathbf{M} = (104802.05, 92315.11), \qquad \mathbf{C} = (98990.71, 88683.78) \text{ ft.}$$

The candidate arc endpoint errors are 13705.23 ft and 1.02 ft. The second candidate is selected:

$$\mathbf{Q} = (104575.87, 84713.36) \text{ ft}, \qquad \theta_o = 54.6°, \qquad \mathbf{U} = (100664.63, 79209.71) \text{ ft.}$$

The briefing shows `POP BRG 99.8 deg M` and `POP RNG 3.4 NM`; the action-point `Offset` is $\omega = |54.6 - 82| = 27.4°$. Support offsets are `PUP BRG 271.8 / RNG 20801 ft` and `OA1 BRG 122 / RNG 2647 ft`, both true. These coordinates and variation are test inputs, not BMS-file samples.

Code. `templates/dtc.html`, `getTurnCenterAtFinalPoint`, the right-normal construction of $\mathbf{C}$:

```
816 const radians = normalizeBearing(finalHeading) * Math.PI / 180;
817 const rightNormal = {
818   northFeet: -Math.sin(radians),
819   eastFeet: Math.cos(radians),
820 };
821 return turnDirection === "left"
822   ? {
823     northFeet: finalCoord.northFeet - rightNormal.northFeet * radiusFeet,
824     eastFeet: finalCoord.eastFeet - rightNormal.eastFeet * radiusFeet,
825   }
826   : {
827     northFeet: finalCoord.northFeet + rightNormal.northFeet * radiusFeet,
828     eastFeet: finalCoord.eastFeet + rightNormal.eastFeet * radiusFeet,
829   };
```

The same file's `getCircleTangentsFromPoint` produces $\mathbf{Q}_{\pm}$; `solvePopupTurnGeometry` ranks the resulting arc endpoints. Here `dx` is east and `dy` is north:

```
832 function getCircleTangentsFromPoint(pointCoord, centerCoord, radiusFeet) {
833   if (!pointCoord || !centerCoord || !Number.isFinite(radiusFeet) || radiusFeet <= 0) return [];
834   const dx = pointCoord.eastFeet - centerCoord.eastFeet;
835   const dy = pointCoord.northFeet - centerCoord.northFeet;
836   const distanceSquared = dx * dx + dy * dy;
837   if (distanceSquared <= radiusFeet * radiusFeet) return [];
838   const root = Math.sqrt(distanceSquared - radiusFeet * radiusFeet);
839   return [1, -1].map((sign) => {
840     const eastFeet = centerCoord.eastFeet + (radiusFeet * radiusFeet * dx + sign * radiusFeet * root * -dy) /
       distanceSquared;
841     const northFeet = centerCoord.northFeet + (radiusFeet * radiusFeet * dy + sign * radiusFeet * root * dx) /
       distanceSquared;
842     return { northFeet, eastFeet };
843   });
844 }
845
846 function solvePopupTurnGeometry(actionCoord, mapCoord, attackHeading, turnRadius, turnDirection) {
847   const centerCoord = getTurnCenterAtFinalPoint(mapCoord, attackHeading, turnRadius, turnDirection);
848   if (!centerCoord) return null;
849   const candidates = getCircleTangentsFromPoint(actionCoord, centerCoord, turnRadius)
850     .map((pdpCoord) => {
851       const offsetLegMetrics = bearingRangeBetween(actionCoord, pdpCoord);
852       if (!offsetLegMetrics) return null;
853       const offsetLegHeading = Number.parseFloat(offsetLegMetrics.brg);
854       const turnArcCoords = buildTurnArcCoords(pdpCoord, offsetLegHeading, attackHeading, turnRadius,
      turnDirection);
855       const arcEndCoord = turnArcCoords.length ? turnArcCoords[turnArcCoords.length - 1] : null;
856       return {
857         pdpCoord,
858         offsetLegMetrics,
859         offsetLegHeading,
860         turnArcCoords,
861         errorFeet: coordDistanceFeet(arcEndCoord, mapCoord),
862       };
863     })
864     .filter((candidate) => candidate && Number.isFinite(candidate.errorFeet));
865   if (!candidates.length) return null;
```

```
  candidates.sort((a, b) => a.errorFeet - b.errorFeet);
  return candidates[0];
}
```

`buildTurnArcCoords` in the same file reconstructs the circle from the rounded starting heading and rotates the initial radius. At `index === steps`, the appended point is $\mathbf{A}_j$; `getTurnDelta` computes ψ_j:

```
const turnDelta = getTurnDelta(startHeading, endHeading, turnDirection);
if (!Number.isFinite(turnDelta) || Math.abs(turnDelta) < 0.1) return [startCoord];
const startRadians = normalizeBearing(startHeading) * Math.PI / 180;
const rightNormal = {
  northFeet: -Math.sin(startRadians),
  eastFeet: Math.cos(startRadians),
};
const centerCoord = turnDelta >= 0
  ? {
    northFeet: startCoord.northFeet + rightNormal.northFeet * radiusFeet,
    eastFeet: startCoord.eastFeet + rightNormal.eastFeet * radiusFeet,
  }
  : {
    northFeet: startCoord.northFeet - rightNormal.northFeet * radiusFeet,
    eastFeet: startCoord.eastFeet - rightNormal.eastFeet * radiusFeet,
  };
const relative = {
  northFeet: startCoord.northFeet - centerCoord.northFeet,
  eastFeet: startCoord.eastFeet - centerCoord.eastFeet,
};
const steps = Math.max(8, Math.ceil(Math.abs(turnDelta) / 6));
const points = [];
for (let index = 0; index <= steps; index += 1) {
  const radians = (turnDelta * index / steps) * Math.PI / 180;
  const cos = Math.cos(radians);
  const sin = Math.sin(radians);
  const eastFeet = relative.eastFeet * cos + relative.northFeet * sin;
  const northFeet = -relative.eastFeet * sin + relative.northFeet * cos;
  points.push({
    northFeet: centerCoord.northFeet + northFeet,
    eastFeet: centerCoord.eastFeet + eastFeet,
  });
```

`templates/dtc_popup.js`, `computePopupPlan`, converts the entered heading, constructs the points, and measures the offsets. `offsetLegHeading` is the selected rounded θ_o:

```
const magneticVariation = getPopupMagneticVariation();
// Convert the pilot's heading once; geometry and DTC offset rows stay true.
const trueHeading = normalizeBearing(heading + (magneticVariation ?? 0));
let attackHeading;
let ingressHeading;
if (headingMode === "attack") {
  attackHeading = trueHeading;
  ingressHeading = normalizeBearing(attackHeading + (side === "right" ? -angleOff : angleOff));
} else {
  ingressHeading = trueHeading;
  attackHeading = normalizeBearing(ingressHeading + (side === "right" ? angleOff : -angleOff));
}

const turnDirection = side === "left" ? "left" : "right";
const actionCoord = projectBearingRange(targetCoord, ingressHeading + 180, actionRangeFeet);
const mapCoord = projectBearingRange(targetCoord, attackHeading + 180, mapDistance);
const releaseCoord = projectBearingRange(targetCoord, attackHeading + 180, releaseReferenceRange);
const turnGeometry = solvePopupTurnGeometry(actionCoord, mapCoord, attackHeading, turnRadius, turnDirection);
if (!turnGeometry) return { error: "Could not solve action-to-MAP turn geometry." };
const pdpCoord = turnGeometry.pdpCoord;
const offsetLegMetrics = turnGeometry.offsetLegMetrics;
const offsetLegHeading = turnGeometry.offsetLegHeading;
const offsetLegDistance = coordDistanceFeet(actionCoord, pdpCoord);
if (popToPullDownDistance > offsetLegDistance) {
  return { error: "PUP would fall before the action point. Increase action range or adjust the vertical profile."
      };
```

```
}
const pupCoord = projectBearingRange(pdpCoord, offsetLegHeading + 180, popToPullDownDistance);
const oaCoord = projectBearingRange(targetCoord, attackHeading, aod);
if (!actionCoord || !pdpCoord || !pupCoord || !oaCoord) return { error: "Could not compute pop-up geometry." };
const turnArcCoords = turnGeometry.turnArcCoords;
const rolloutCoord = mapCoord;

const pupMetrics = bearingRangeBetween(selectedCoord, pupCoord);
const oaMetrics = bearingRangeBetween(selectedCoord, oaCoord);
if (!pupMetrics || !oaMetrics) return { error: "Could not compute NAV offset rows." };
const ingressToOffset = Math.abs(normalizeBearing(offsetLegHeading - ingressHeading));
const offsetAngle = ingressToOffset > 180 ? 360 - ingressToOffset : ingressToOffset;
```

References

[1] Jean Meeus, *Astronomical Algorithms*, Willmann–Bell, 1991.

[2] NOAA, NASA and U.S. Air Force, *U.S. Standard Atmosphere, 1976*, NASA-TM-X-74335, 1976, Part 1. https://ntrs.nasa.gov/citations/19770009539.

[3] William S. Aiken, Jr., *Standard Nomenclature for Airspeeds with Tables and Charts for Use in Calculation of Airspeed*, NACA Report 837, 1946. https://ntrs.nasa.gov/api/citations/19930091914/downloads/19930091914.pdf.

[4] Jonathan P. How, *Aircraft Stability and Control*, MIT OpenCourseWare 16.333, Fall 2004, Lecture 1: Equilibrium States, Aircraft Performance, Introduction to Basic Terms. https://ocw.mit.edu/courses/16-333-aircraft-stability-and-control-fall-2004/9befd7ea465e31f9ff763c5587815264_lecture_1.pdf.

[5] oakdesign, *Falcon BMS Tacview Converter*, software repository, src/utils/coordinate_converter.py (create_proj_string and CoordinateConverter). https://github.com/oakdesign/falcon-bms-tacview-converter/.